

DigiDentPlant: Digital Dental Implant

Zahnsegmentierung mit tiefen neuronalen Netzwerken

Bachelorarbeit

ausgearbeitet von

Ali Cemal Deniz

zur Erlangung des akademischen Grades
Bachelor of Science (B.Sc.)

vorgelegt an der

Türkisch-Deutschen Universität
Fakultät für Ingenieurwissenschaften

im Studiengang
Informatik

Betreuer/in: Assist. Prof. Dr. Dilek GÖKSEL DURU
Türkisch-Deutsche Universität

Istanbul, im Juni 2025

Eidesstattliche Erklärung

Ich versichere, die von mir vorgelegte Arbeit selbständig verfasst zu haben.

Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten oder nicht veröffentlichten Arbeiten anderer entnommen sind, habe ich als entnommen kenntlich gemacht. Sämtliche Quellen und Hilfsmittel, die ich für die Arbeit benutzt habe, sind angegeben.

Die Arbeit hat mit gleichem Inhalt bzw. in wesentlichen Teilen noch keiner anderen Prüfungsbehörde vorgelegen.

Istanbul, 23. Juni 2025

Ali Cemal Deniz

Kurzfassung

Die exakte Segmentierung von Zähnen aus dentalen Computertomographie-(CT)-Aufnahmen spielt eine zentrale Rolle bei der Diagnosestellung und Behandlungsplanung in der Zahnmedizin. Herkömmliche Segmentierungsverfahren stoßen jedoch oft an ihre Grenzen, insbesondere bei komplexen Fällen wie fehlenden Zähnen oder vorhandenen prothetischen Versorgungen. Obwohl Deep-Learning-Verfahren zunehmend eingesetzt werden, fehlt es in der bisherigen Literatur an umfassenden vergleichenden Studien zu verschiedenen Segmentierungsmodellen speziell für dentale CT-Bilder. Ziel dieser Arbeit ist es, diese Forschungslücke durch einen systematischen Vergleich mehrerer Deep-Learning-Methoden wie UNet, SegNet, BASNet, BiSeNet, DenseASPP und Mask R-CNN zu schließen. Die Leistungsfähigkeit der Modelle wurde dabei anhand eines eigens erstellten 2D-Datensatzes quantitativ bewertet, wobei der Dice Similarity Coefficient (DSC), Precision, Recall sowie der F1-Score als zentrale Evaluationsmetriken genutzt wurden. Die Ergebnisse ermöglichen eine klare Einordnung der verschiedenen Modelle und bieten somit eine wertvolle Orientierungshilfe für Kliniker und Forscher bei der Auswahl des optimalen Deep-Learning-Modells für die klinische Anwendung. Insgesamt schafft diese Studie eine grundlegende Referenz für zukünftige Forschungen im Bereich der dentalen Bildgebung und Segmentierung.

Schlüsselwörter: Zahnsegmentierung, Deep Learning, Convolutional Neural Networks, Mask R-CNN, BASNet, SegNet, DenseASPP, U-Net, Computertomographie, Vergleich.

Abstract

Accurate segmentation of teeth from dental computed tomography (CT) images plays a crucial role in clinical diagnosis and treatment planning in dentistry. However, conventional segmentation methods often reach their limits, especially in complex cases involving missing teeth or existing prosthetic restorations. While deep learning methods are increasingly being utilized, comprehensive comparative studies of various segmentation models specifically for dental CT images remain scarce. The aim of this study is to address this research gap by systematically comparing several deep learning architectures, including UNet, SegNet, BASNet, BiSeNet, DenseASPP, and Mask R-CNN. The performance of these models was quantitatively evaluated using a self-created 2D dataset, employing the Dice Similarity Coefficient (DSC), precision, recall, and F1-score as primary evaluation metrics. The results facilitate a clear assessment of the different models, providing valuable guidance for clinicians and researchers in selecting the optimal deep learning model for clinical practice. Overall, this study establishes a foundational reference for future research in the field of dental imaging and segmentation.

Keywords: Teeth Segmentation, Deep Learning, Convolutional Neural Networks, Mask R-CNN, BASNet, SegNet, DenseASPP, U-Net, Tomography, Comparison.

Inhaltsverzeichnis

Eidesstattliche Erklärung **2**

Abstract **3**

Abbildungsverzeichnis **3**

Tabellenverzeichnis **4**

1. Einleitung **5**

2. Grundlagen **7**
2.1. Grundlagen der Bildsegmentierung 7
2.1.1. Definition der Segmentierung 7
2.1.2. Herausforderungen in der medizinischen Segmentierung 8
2.1.3. Klassische Segmentierungsmethoden 9
2.1.4. Bildsegmentierung mittels Künstlicher Intelligenz 10
2.2. Maschinelles Lernen und Deep Learning 11
2.2.1. Einführung in das Maschinelle Lernen 11
2.2.2. Grundlagen neuronaler Netzwerke 12
2.2.3. Aufbau und Funktionsweise 13
2.2.4. Convolutional Neural Networks (CNNs) 18
2.3. Evaluationsmetriken für Segmentierungsaufgaben 25
2.3.1. Dice Similarity Coefficient (DSC) 25
2.3.2. Precision, Recall und F1-Score 25

3. Stand der Technik **27**

4. Methoden und Ergebnisse **29**
4.1. Experimenteller Aufbau und Datensatz 29
4.1.1. Experimentumgebung 29
4.1.2. Datensatz 31
4.2. Verwendete Methoden 33
4.2.1. Verwendete Deep-Learning-Modelle 33
4.2.2. Optimierung der Modelle 42
4.3. Experimente 51
4.3.1. Durchführung der Experimente 51
4.3.2. Evaluationsmetriken 52
4.3.3. Ergebnisse der Experimente 54
4.3.4. Diskussion der Ergebnisse 66
4.4. Fazit und zukünftige Arbeiten 71

Anhang A. Anhang: Lernkurven **73**

Anhang B. Anhang: Dice Similarity Coefficient (DSC) **76**

Literaturverzeichnis **80**

Abbildungsverzeichnis

2.1. Schematischer Aufbau eines künstlichen neuronalen Netzwerks [Géron, 2022]. 13
2.2. Funktionsweise eines künstlichen Neurons [Planche and Andres, 2019]. . 15
2.3. Illustration des Gradientenabstiegsverfahrens zur Minimierung der Verlustfunktion [Géron, 2022]. 17
2.4. Typischer Aufbau eines Convolutional Neural Networks [Géron, 2022]. . 19
2.5. Veranschaulichung der Convolution-Operation mit einer Schrittweite von 2 [Géron, 2022]. 20
2.6. Veranschaulichung von Max-Pooling [Géron, 2022]. 21
2.7. Visualisierung der Dropout-Technik [Géron, 2022]. 23
2.8. Schematischer Aufbau des U-Net Architektur [Planche and Andres, 2019]. 24

4.1. Architektur des UNet-Modells [Ronneberger et al., 2015]. 33
4.2. Architektur des BiSeNetV2-Modells [Yu et al., 2021]. 35
4.3. Architektur des DenseASPP-Modells [Yang et al., 2018]. 36
4.4. Architektur des SegNet-Modells [Badrinarayanan et al., 2017]. 38
4.5. Architektur des BASNet-Modells [Qin et al., 2021]. 39
4.6. Architektur des Mask R-CNN-Modells [He et al., 2017]. 41
4.7. Schematische Darstellung der verwendeten Fine-Tuning-Strategie. . . . 49
4.8. Beispielhafte Segmentierungsergebnisse des UNet-Modells 55
4.9. Beispielhafte Segmentierungsergebnisse des UNet-Modells 55
4.10. Beispielhafte Segmentierungsergebnisse des UNet-Modells 56
4.11. Beispielhafte Segmentierungsergebnisse des UNet-Modells 56
4.12. Beispielhafte Segmentierungsergebnisse des UNet-Modells 56
4.13. Beispielhafte Segmentierungsergebnisse des SegNet-Modells 57
4.14. Beispielhafte Segmentierungsergebnisse des SegNet-Modells 57
4.15. Beispielhafte Segmentierungsergebnisse des SegNet-Modells 58
4.16. Beispielhafte Segmentierungsergebnisse des SegNet-Modells 58
4.17. Beispielhafte Segmentierungsergebnisse des SegNet-Modells 58
4.18. Beispielhafte Segmentierungsergebnisse des BASNet-Modells 59
4.19. eispielhafte Segmentierungsergebnisse des BASNet-Modells 59

4.20. Beispielhafte Segmentierungsergebnisse des BASNet-Modells 60
4.21. Beispielhafte Segmentierungsergebnisse des BASNet-Modells 60
4.22. Beispielhafte Segmentierungsergebnisse des BASNet-Modells 60
4.23. Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells 61
4.24. Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells 61
4.25. Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells 62
4.26. Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells 62
4.27. Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells 62
4.28. Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells 63
4.29. Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells 63
4.30. Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells 64
4.31. Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells 64
4.32. Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells 64

A.1. Lernkurve des Unet-Modell 73
A.2. Lernkurve des UNet-Modells mit Fine-Tunning 73
A.3. Lernkurve des SegNet-Modells 74
A.4. Lernkurve des BASNet-Modells 74
A.5. Lernkurve des BiSeNetV2-Modells 75
A.6. Lernkurve des DenseASPP-Modells 75

B.1. Dice Similarity Coefficient (DSC) des Unet-Modell 76
B.2. Dice Similarity Coefficient (DSC) des UNet-Modells mit Fine-Tunning . 76
B.3. Dice Similarity Coefficient (DSC) des SegNet-Modells 77
B.4. Dice Similarity Coefficient (DSC) des BASNet-Modells 77
B.5. Dice Similarity Coefficient (DSC) des BiSeNetV2-Modells 78
B.6. Dice Similarity Coefficient (DSC) des DenseASPP-Modells 78

Tabellenverzeichnis

4.1. Verwendete Softwarebibliotheken und Versionen 30
4.2. Technische Spezifikationen des Computersystems 30
4.3. Quantitative Ergebnisse der getesteten Modelle 54

1. Einleitung

Die präzise Segmentierung dentaler Strukturen aus Computertomographie-(CT)-Aufnahmen spielt eine zentrale Rolle in der modernen Zahnmedizin. Außerdem bildet sie die Grundlage zahlreicher klinischer Anwendungen, wie zum Beispiel der präzisen Diagnostik, der Behandlungsplanung und der patientenspezifischen Therapieplanung. Eine exakte Segmentierung ermöglicht es, anatomische Strukturen eindeutig voneinander und vom Hintergrund abzugrenzen, was insbesondere bei komplexen klinischen Situationen wie Zahnverlust oder vorhandenen prothetischen Versorgungen entscheidend ist.

Traditionelle Bildsegmentierungsmethoden, beispielsweise Schwellenwertverfahren, Region Growing oder Watershed-Algorithmen, stoßen jedoch häufig an ihre Grenzen, wie zum Beispiel bei der Verarbeitung medizinischer Bilddaten mit inhomogenen Intensitäten, anatomischer Variabilität und Bildartefakten. Die manuelle Segmentierung hingegen ist zeitintensiv, kostenintensiv und anfällig für inter- und intraindividuelle Variabilität.

In den letzten Jahren haben Deep-Learning-Ansätze, insbesondere Convolutional Neural Networks (CNNs), erhebliche Fortschritte in der automatisierten Bildsegmentierung erzielt. Im Gegensatz zu klassischen Verfahren sind diese neuronalen Netzwerke in der Lage, relevante Merkmale direkt aus umfangreichen Trainingsdaten zu lernen. Außerdem können sie komplexe anatomische Strukturen mit hoher Genauigkeit segmentieren. Trotz der zunehmenden Verwendung von Deep-Learning-Modellen in der medizinischen Bildgebung gibt es derzeit nur wenige umfassende Studien, die verschiedene Modelle speziell für die Segmentierung zahnmedizinischer Bilder vergleichend bewerten.

Diese Forschungslücke bildet den Ausgangspunkt der vorliegenden Arbeit. Ziel dieser Studie ist es, eine systematische Evaluierung und einen Vergleich verschiedener moderner Deep-Learning-Architekturen durchzuführen, darunter UNet, SegNet, BASNet, BiSeNet, DenseASPP und Mask R-CNN, und deren Leistungsfähigkeit auf einem eigens erstellten dentalen 2D-CT-Datensatz quantitativ zu bewerten. Die erzielten Ergebnisse sollen eine solide Grundlage für die klinische Praxis und zukünftige Forschung bieten. Außerdem können sie Klinikern und Forschern bei der Auswahl des am besten geeigneten Modells helfen.

Im weiteren Verlauf dieser Arbeit werden zunächst grundlegende theoretische Konzepte zur Bildsegmentierung und zu Deep Learning erläutert. Bevor im methodischen Teil die verwendeten Modelle, der Datensatz, der Trainingsprozess und die Evaluationsmetriken detailliert beschrieben werden. Abschließend erfolgt eine umfassende Diskussion der Ergebnisse sowie ein Ausblick auf mögliche zukünftige Forschungsansätze.

2. Grundlagen

In diesem Kapitel werden die für das Verständnis der Arbeit notwendigen theoretischen Grundlagen dargestellt. Dazu gehören grundlegende Konzepte der medizinischen Bildgebung und Bildsegmentierung sowie die wichtigsten Prinzipien des maschinellen Lernens und Deep Learnings. Zudem erfolgt eine Einführung in spezifische neuronale Netzwerkarchitekturen und Evaluationsmetriken, die im Rahmen der vorliegenden Studie verwendet wurden.

2.1. Grundlagen der Bildsegmentierung

In diesem Abschnitt werden die grundlegenden Prinzipien der Bildsegmentierung erläutert. Dazu gehören eine allgemeine Definition und typische Herausforderungen, denen speziell in der medizinischen Bildsegmentierung begegnet wird. Weiterhin erfolgt ein Überblick über klassische Segmentierungsmethoden sowie moderne Ansätze basierend auf Künstlicher Intelligenz.

2.1.1. Definition der Segmentierung

Bildsegmentierung bezeichnet den Prozess der Unterteilung eines Bildes in mehrere disjunkte Teilbereiche, sogenannte Segmente, die hinsichtlich definierter Merkmale (wie Intensität, Farbe oder Textur) homogen sind. In der medizinischen Bildverarbeitung dient die Segmentierung insbesondere dazu, anatomische Strukturen oder pathologische Bereiche eindeutig voneinander und vom Hintergrund zu unterscheiden. Ziel ist es, medizinisch relevante Bildregionen automatisch zu identifizieren und quantitativ zu analysieren, um Diagnosen und Behandlungen zu unterstützen [Pham et al., 2000].

Formal betrachtet kann die Segmentierung eines Bildes als Zerlegung des gesamten Bildbereichs R in Teilbereiche R_i dargestellt werden, wobei folgende Bedingungen erfüllt sein müssen (2.1) [Pal and Pal, 1993]:

$$R = \bigcup_{i=1}^{n} R_i, \quad \text{mit} \quad R_i \cap R_j = \emptyset \quad \text{für alle } i \neq j \tag{2.1}$$

Dabei repräsentiert jedes Segment R_i einen spezifischen Bildbereich, der beispielsweise einer anatomischen Struktur oder einer definierten Objektklasse entspricht [Pal and Pal, 1993].

2.1.2. Herausforderungen in der medizinischen Segmentierung

Die Segmentierung medizinischer Bilddaten ist mit vielfältigen Herausforderungen verbunden, die sich insbesondere aus der Komplexität anatomischer Strukturen und der Qualität der Bilddaten ergeben. Häufige Herausforderungen umfassen:

- **Bildrauschen und Artefakte**: Medizinische Aufnahmen wie CT oder MRT enthalten häufig Bildrauschen und Artefakte, die aus Messverfahren, Patientenbewegungen oder Gerätefehlern resultieren. Dies erschwert eine exakte Segmentierung deutlich.
- **Inhomogene Intensitäten**: Anatomische Strukturen weisen oft keine einheitliche Intensität auf, was eine klare Abgrenzung der Segmente erschwert.
- **Fehlende oder diffuse Grenzen**: Viele anatomische Strukturen besitzen keine klar definierte Grenze zu benachbarten Geweben. Die exakte Trennung wird dadurch zur Herausforderung.
- **Anatomische Variabilität**: Hohe Variabilität zwischen unterschiedlichen Patienten, wie Größe, Form oder Lage anatomischer Strukturen, erfordert robuste und generalisierbare Segmentierungsmethoden.
- **Pathologische Veränderungen**: Erkrankungen oder pathologische Veränderungen können die normale Anatomie stark verändern und erschweren so zusätzlich die Segmentierung.
- **Begrenzte Menge annotierter Daten**: Die manuelle Erstellung von Segmentierungsmasken ist zeitaufwendig und teuer, wodurch oft nur eine begrenzte Menge an Trainingsdaten zur Verfügung steht.

Aufgrund dieser Herausforderungen erfordern medizinische Segmentierungsverfahren spezialisierte Methoden und robuste Algorithmen, welche insbesondere durch den Einsatz moderner Deep-Learning-Ansätze zunehmend realisiert werden [Pham et al., 2000; Minaee et al., 2021].

2.1.3. Klassische Segmentierungsmethoden

Vor dem Einsatz moderner Deep-Learning-Methoden wurden in der medizinischen Bildsegmentierung überwiegend klassische Methoden verwendet, die auf expliziten Regeln und mathematischen Verfahren basieren. Zu den bekanntesten klassischen Segmentierungsverfahren gehören:

2.1.3.1. Schwellenwertverfahren (Thresholding)

Beim Schwellenwertverfahren (2.2) wird eine Intensitätsschwelle T definiert, um Bildpixel anhand ihres Intensitätswertes in Vorder- und Hintergrund zu unterteilen. Die Segmentierung erfolgt somit pixelweise nach der Regel [Pal and Pal, 1993]:

$$f(x,y) = \begin{cases} 1, & \text{wenn } I(x,y) \geq T \\ 0, & \text{wenn } I(x,y) < T \end{cases} \tag{2.2}$$

Hierbei (2.2) steht $I(x,y)$ für die Intensität des Pixels an der Position (x,y). Schwellenwertverfahren sind einfach und recheneffizient, haben jedoch Probleme bei inhomogenen Intensitäten und stark verrauschten Bildern [Pal and Pal, 1993].

2.1.3.2. Region Growing

Region Growing ist ein Segmentierungsverfahren, das von sogenannten „Seed-Punkten“ ausgehend benachbarte Pixel mit ähnlichen Intensitätswerten zu homogenen Regionen zusammenfügt. Die Region wächst iterativ, indem benachbarte Pixel aufgenommen werden, solange eine definierte Ähnlichkeitsbedingung erfüllt ist (2.3) [Pham et al., 2000]:

$$|I(x,y) - I_{\text{seed}}| < \epsilon \tag{2.3}$$

Hierbei (2.3) steht I_{seed} für den Intensitätswert des Seed-Pixels und ϵ für eine vorab definierte Toleranzschwelle. Region Growing ist robust gegenüber lokalen Intensitätsschwankungen, erfordert jedoch sorgfältig ausgewählte Seed-Punkte und Schwellwerte [Pham et al., 2000].

2.1.3.3. Watershed-Algorithmus

Der Watershed-Algorithmus basiert auf einer topographischen Interpretation des Bildes, bei der Intensitätswerte als Höheninformation betrachtet werden [Pham et al., 2000]. Dabei werden Pixel als Einzugsgebiete interpretiert, und die Segmentgrenzen

entstehen an sogenannten „Wasserscheiden“ zwischen diesen Regionen. Dieses Verfahren ist besonders geeignet, um zusammenhängende Strukturen effektiv zu segmentieren, neigt aber dazu, Regionen zu übersegmentieren, insbesondere bei verrauschten Daten. Verbesserungen wie markerbasierte Watershed-Methoden wurden entwickelt, um Übersegmentierungen entgegenzuwirken [Pham et al., 2000].

Klassische Methoden sind nach wie vor relevant, insbesondere als Grundlage oder Initialisierung für komplexere Verfahren wie Deep-Learning-Ansätze, da sie intuitiv verständlich und häufig rechneffizient sind.

2.1.4. Bildsegmentierung mittels Künstlicher Intelligenz

In den letzten Jahren haben Methoden der künstlichen Intelligenz (KI), insbesondere Deep-Learning-Ansätze, in der medizinischen Bildsegmentierung erhebliche Fortschritte erzielt. Im Gegensatz zu klassischen Verfahren lernen KI-basierte Modelle relevante Merkmale direkt aus umfangreichen Datensätzen und können somit komplexe und heterogene anatomische Strukturen automatisch erkennen.

Ein häufig verwendeter Ansatz zur KI-basierten Bildsegmentierung ist der Einsatz von Convolutional Neural Networks (CNNs). Diese Netzwerke können räumliche und strukturelle Merkmale aus medizinischen Bilddaten automatisch extrahieren und nutzen speziell entwickelte Architekturen, wie Fully Convolutional Networks (FCN), U-Net, SegNet oder Mask R-CNN, um pixelgenaue Segmentierungen zu generieren.

Der wesentliche Vorteil dieser KI-Methoden liegt in ihrer Fähigkeit, auch feine und schwer abgrenzbare Strukturen sowie anatomische Variationen zuverlässig zu segmentieren. Gleichzeitig benötigen diese Verfahren jedoch große Mengen qualitativ hochwertiger, annotierter Trainingsdaten sowie erhebliche Rechenressourcen für das Modelltraining [Minaee et al., 2021; Ronneberger et al., 2015].

Dank ihrer hervorragenden Leistungsfähigkeit und ihrer Flexibilität werden KI-basierten Segmentierungsmethoden zunehmend als Standard in der medizinischen Diagnostik und Bildanalyse etabliert.

2.2. Maschinelles Lernen und Deep Learning

In diesem Abschnitt werden die Grundlagen und wichtigsten Konzepte des maschinellen Lernens und Deep Learnings vorgestellt. Hierbei liegt der Fokus auf dem Aufbau und der Funktionsweise künstlicher neuronaler Netzwerke, insbesondere Convolutional Neural Networks (CNNs). Zudem werden zentrale Themen wie der Trainingsprozess, gängige Verlustfunktionen und Optimierungsverfahren erläutert.

2.2.1. Einführung in das Maschinelle Lernen

Maschinelles Lernen ist ein Bereich, der die Grundlage der künstlichen Intelligenz bildet und darauf basiert, dass der Computer versucht, bestimmte Muster vorherzusagen. Je nach verfügbarer Datenstruktur und Lernziel unterscheidet man hauptsächlich drei Lernstrategien:

- **Überwachtes Lernen (Supervised Learning)**: Dabei existieren zu den Eingabedaten x bereits bekannte Zielgrößen y. Ziel ist es, eine Funktion $f(x)$ zu finden, welche die Beziehung zwischen Eingaben und Zielgrößen optimal beschreibt. Mathematisch (2.4) [Géron, 2022]:

$$y = f(x) + \epsilon \tag{2.4}$$

  wobei ϵ einen Fehlerterm darstellt.

  Typische Aufgaben sind Regression (z.B. lineare Regression) und Klassifikation.

- **Unüberwachtes Lernen (Unsupervised Learning)**: Hierbei sind keine Zielgrößen vorhanden, und das Ziel ist, inhärente Strukturen oder Muster in den Daten zu entdecken. Bekannte Methoden sind Clustering (z.B. k-Means) und Dimensionsreduktion (z.B. PCA) [Géron, 2022].

- **Teilüberwachtes Lernen (Semi-Supervised Learning)**: Diese Methode kombiniert die beiden zuvor genannten Ansätze, indem sowohl beschriftete als auch unbeschriftete Daten verwendet werden. Ziel ist es, trotz einer geringen Anzahl beschrifteter Daten eine hohe Lernleistung zu erreichen [Géron, 2022].

Die Güte eines Modells beim überwachten Lernen wird typischerweise mit einer Kostenfunktion (*Loss-Funktion*) quantifiziert, beispielsweise dem mittleren quadratischen Fehler (*Mean Squared Error*, MSE) (2.5) [Géron, 2022]:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - f(x_i))^2 \tag{2.5}$$

Der Trainingsprozess besteht darin, Modellparameter so anzupassen, dass die Kostenfunktion minimiert wird.

Maschinelles Lernen ist insbesondere bei komplexen Datenanalysen wie der medizinischen Bildverarbeitung, Spracherkennung und autonomen Systemen sehr erfolgreich [Géron, 2022].

2.2.2. Grundlagen neuronaler Netzwerke

Künstliche neuronale Netzwerke stellen heute ein Kernstück vieler moderner Verfahren im maschinellen Lernen dar. Ihr Aufbau orientiert sich lose an der Struktur natürlicher neuronaler Systeme. Solche Netzwerke bestehen aus zahlreichen Verarbeitungsknoten, den sogenannten künstlichen Neuronen, welche untereinander vernetzt sind. Diese Neuronen gruppieren sich typischerweise in mehreren aufeinanderfolgenden Ebenen, die üblicherweise aus einer Eingangsebene (Input Layer), einer oder mehreren Zwischenschichten (Hidden Layers) sowie einer abschließenden Ausgabeschicht (Output Layer) bestehen.

Jedes künstliche Neuron erhält Eingaben x_i, multipliziert diese mit Gewichten w_i und summiert sie anschließend mit einem zusätzlichen Bias-Term b. Die gewichtete Summe z ist definiert als (2.6) [Géron, 2022]:

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{2.6}$$

Um komplexe, nichtlineare Zusammenhänge zu modellieren, wird die berechnete Summe z durch eine Aktivierungsfunktion $f(z)$ transformiert (2.6) [Géron, 2022]:

$$a = f(z) \tag{2.7}$$

Typische Aktivierungsfunktionen sind beispielsweise (2.8) (2.9) (2.10) [Géron, 2022]:

- Rectified Linear Unit (ReLU) (2.8) :

$$f(z) = \max(0, z) \tag{2.8}$$

- Sigmoid-Funktion (2.9):

$$f(z) = \frac{1}{1 + e^{-z}} \tag{2.9}$$

- Softmax-Funktion (für Klassifikationsprobleme mit mehreren Klassen) (2.10):

$$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}) \tag{2.10}$$

Das Training neuronaler Netzwerke (*Siehe Abbildung 2.1*) besteht darin, die Netzwerkparameter (Gewichte und Bias) iterativ anzupassen [Géron, 2022]. Dies geschieht mittels der Minimierung einer Verlustfunktion und eines Optimierungsverfahrens, typischerweise durch Gradientenabstieg (Gradient Descent). Ein weitverbreitetes Verfahren zur Berechnung der Gradienten ist der Backpropagation-Algorithmus, bei dem Fehler von der Ausgabeschicht rückwärts durch das Netzwerk propagiert werden [Géron, 2022].

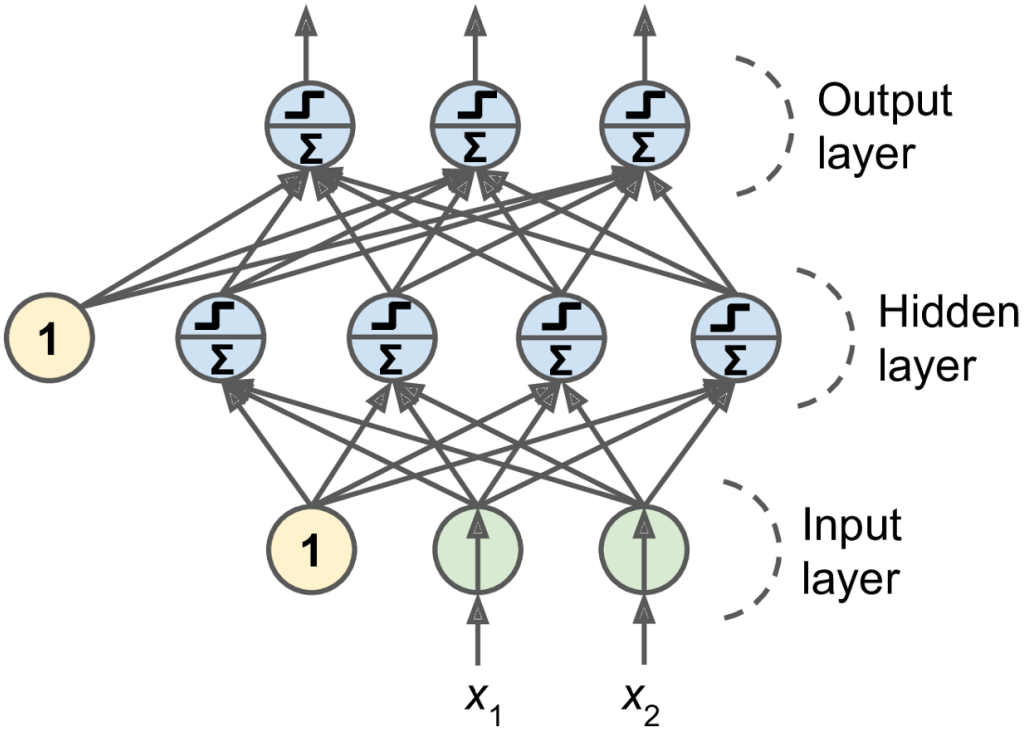


Abbildung 2.1.: Schematischer Aufbau eines künstlichen neuronalen Netzwerks [Géron, 2022].

Neuronale Netzwerke haben sich insbesondere bei komplexen Mustererkennungsaufgaben wie Bildverarbeitung, Spracherkennung und medizinischer Diagnostik bewährt und ermöglichen eine effiziente Verarbeitung umfangreicher, hochdimensionaler Datensätze [Géron, 2022].

2.2.3. Aufbau und Funktionsweise

Ein künstliches neuronales Netzwerk ist eine Netzwerkstruktur, die durch die Kombination künstlicher Neuronen in Schichten gebildet wird [Géron, 2022]. Diese umfassen typischerweise eine Eingabeschicht (*Input Layer*), eine oder mehrere verborgene Schichten (*Hidden Layers*) und eine Ausgabeschicht (*Output Layer*) [Géron, 2022].

Ein künstliches Neuron (*Siehe Abbildung 2.2*) erhält mehrere Eingangssignale $x_1, x_2, \ldots, x_n$, die jeweils mit Gewichten $w_1, w_2, \ldots, w_n$ versehen sind. Zusätzlich besitzt jedes Neuron einen Bias-Wert b. Die Eingabesignale werden zunächst zu einer gewichteten Summe zusammengefasst (2.6) [Géron, 2022]:

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{2.6}$$

Diese errechnete Summe z wird anschließend mithilfe einer nichtlinearen Aktivierungsfunktion $f(\cdot)$ verarbeitet, wodurch das Netzwerk in der Lage ist, komplexere, nichtlineare Zusammenhänge zu erfassen (2.7) [Géron, 2022]:

$$a = f(z) \tag{2.7}$$

Die Wahl der Aktivierungsfunktion $f(z)$ beeinflusst maßgeblich die Lernfähigkeit und Effizienz des neuronalen Netzwerks. Häufig genutzte Aktivierungsfunktionen sind unter anderem:

- **Rectified Linear Unit (ReLU)** (2.8):

$$f(z) = \max(0, z) \tag{2.8}$$

- **Sigmoid-Funktion** (2.9):

$$f(z) = \frac{1}{1 + e^{-z}} \tag{2.9}$$

- **Tanh-Funktion (Tangens Hyperbolicus)** (2.11):

$$f(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} \tag{2.11}$$

Im Trainingsprozess des neuronalen Netzwerks werden Gewichte w_i und Bias-Werte b iterativ angepasst, um eine definierte Verlustfunktion zu minimieren . Die Anpassungen erfolgen dabei meist mittels Gradientenabstieg, der über den Backpropagation-Algorithmus durchgeführt wird.

Durch diesen Aufbau und die Funktionsweise sind neuronale Netzwerke in der Lage, komplexe Muster zu erkennen und effektiv auf eine Vielzahl von Aufgabenstellungen anzuwenden [Géron, 2022].

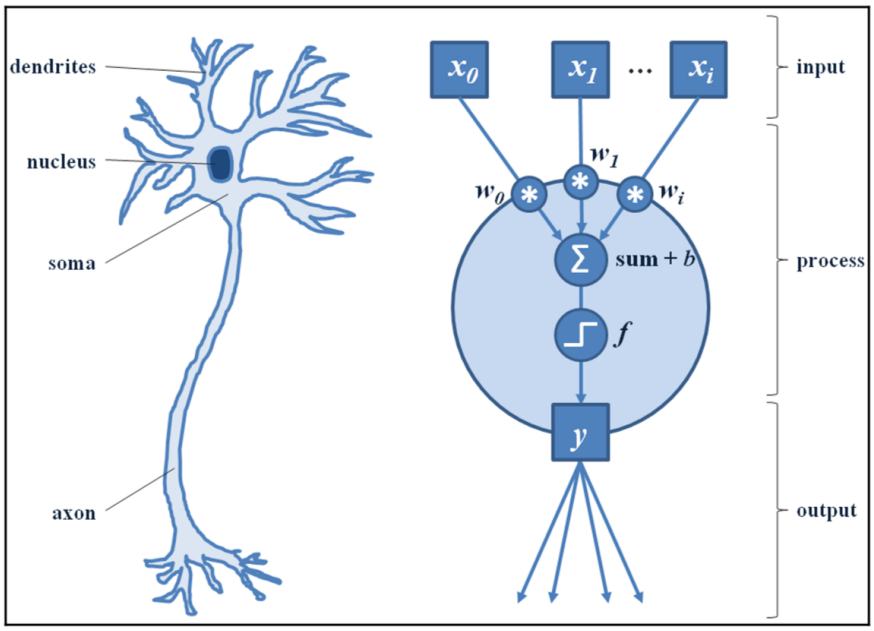


Abbildung 2.2.: Funktionsweise eines künstlichen Neurons [Planche and Andres, 2019].

2.2.3.1. Trainingsprozess, Verlustfunktion (Loss), Optimierung

Das Training künstlicher neuronaler Netzwerke erfolgt iterativ durch Anpassung ihrer Parameter, bestehend aus den Gewichten (w) und den Bias-Werten (b).Dieser iterative Optimierungsprozess zielt darauf ab, die Genauigkeit der Modellvorhersagen und somit die Gesamtleistung kontinuierlich zu steigern. Um die Modellparameter gezielt anzupassen, kommt eine Verlustfunktion (*Loss-Funktion*) (2.5) zum Einsatz, welche den Unterschied zwischen den tatsächlichen Zielwerten und den vom Modell prognostizierten Werten quantitativ bestimmt [Géron, 2022]. Durch die systematische Minimierung dieser Verlustfunktion mittels Optimierungsverfahren, wie beispielsweise dem Gradientenabstieg, lernt das Netzwerk schrittweise, komplexe Zusammenhänge in den Daten exakt und robust abzubilden

Eine häufig verwendete Verlustfunktion bei Regressionsproblemen ist der mittlere quadratische Fehler (*Mean Squared Error, MSE*) (2.5) [Géron, 2022]:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{2.5}$$

Hierbei ist y_i der tatsächliche Wert, $\hat{y}_i$ der vorhergesagte Wert, und n die Anzahl der Trainingsdaten.

Für Klassifikationsprobleme wird oft die Kreuzentropie (*Cross-Entropy Loss*) verwendet, die sich folgendermaßen definiert (2.12) [Géron, 2022]:

$$H(p, q) = -\sum_{i=1}^{n} p_i \log(q_i) \tag{2.12}$$

Dabei bezeichnet p_i die tatsächliche Klassenverteilung und q_i die vom Modell vorhergesagte Wahrscheinlichkeitsverteilung.

Zur Minimierung dieser Verlustfunktion kommt typischerweise das Gradientenabstiegsverfahren (*Gradient Descent*) (2.13) (*Siehe Abbildung 2.3*) zum Einsatz. Hierbei werden die Gewichte iterativ aktualisiert [Géron, 2022]:

$$w_{\text{neu}} = w_{\text{alt}} - \eta\frac{\partial L}{\partial w} \tag{2.13}$$

η steht hierbei (2.13) für die Lernrate, und $\frac{\partial L}{\partial w}$ für den Gradienten der Verlustfunktion bezüglich der Gewichte.

Modernere Optimierungsverfahren wie *Adam*, *RMSProp* oder *Adagrad* adaptieren die Lernrate dynamisch, um Konvergenz und Effizienz des Trainingsprozesses zu verbessern.

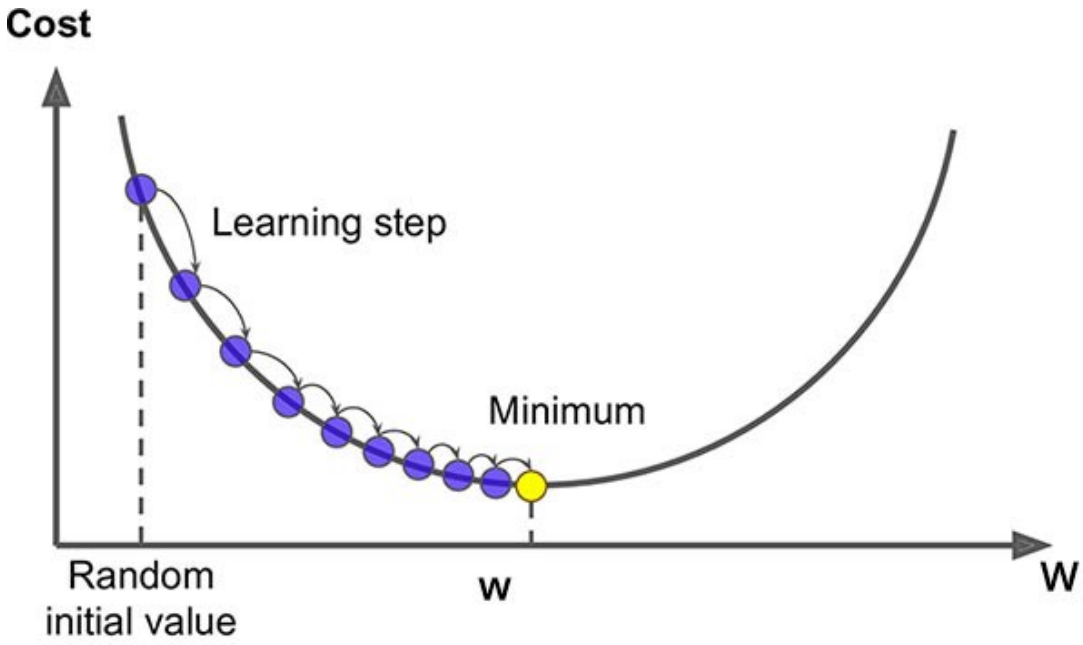


Abbildung 2.3.: Illustration des Gradientenabstiegsverfahrens zur Minimierung der Verlustfunktion [Géron, 2022].

2.2.4. Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs), auch Faltungsnetzwerke genannt, sind eine spezialisierte Form künstlicher neuronaler Netzwerke, die besonders effizient bei der Verarbeitung von Bild- und Videodaten eingesetzt werden. CNNs nutzen die räumliche Struktur der Daten aus und sind in der Lage, automatisch relevante Merkmale (*Features*) aus Eingangsdaten zu extrahieren.

Ein CNN besteht typischerweise aus folgenden zentralen Komponenten:

- **Convolutional Layer (Faltungsschicht)**
- **Pooling Layer (Subsampling-Schicht)**
- **Fully Connected Layer (vollständig verbundene Schicht)**

Die grundlegende Operation in einer Convolutional Layer ist die Faltung (engl. *convolution*). Dabei wird ein kleiner Filter (Kernel) über das Eingangssignal geschoben und berechnet lokale Merkmale durch gewichtete Summen (2.14). Mathematisch lässt sich dies wie folgt ausdrücken (2.14) [Planche and Andres, 2019]:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n) \tag{2.14}$$

Hierbei (2.14) steht I für das Eingabebild (bzw. Eingabematrix), K für den Filter (Kernel) und S für die resultierende sogenannte Feature Map.

Die darauffolgende Pooling-Schicht reduziert die räumliche Größe der Feature Maps, indem lokale Merkmale aggregiert werden. Ein häufig eingesetztes Verfahren ist das Max-Pooling (2.16), bei dem das Maximum innerhalb einer definierten Region selektiert wird. Mathematisch [Planche and Andres, 2019]:

$$P_{max}(R) = \max_{(i,j) \in R} S(i,j) \tag{2.15}$$

Dabei (2.16) bezeichnet R den betrachteten lokalen Bereich der Feature Map.

Nach mehreren aufeinanderfolgenden Convolution- und Pooling-Ebenen schließen sich typischerweise vollständig verknüpfte Schichten (Fully Connected Layers) an, welche dann die abschließende Klassifikation oder Regression übernehmen.

CNNs verwenden üblicherweise nichtlineare Aktivierungsfunktionen, insbesondere ReLU, um die komplexen Zusammenhänge in Daten besser modellieren zu können. Die ReLU-Funktion (2.8) ist definiert als [Géron, 2022]:

$$f(z) = \max(0, z) \tag{2.8}$$

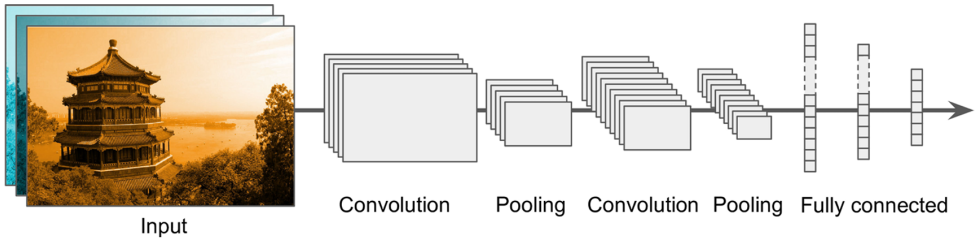


Abbildung 2.4.: Typischer Aufbau eines Convolutional Neural Networks [Géron, 2022].

Convolutional Neural Networks haben insbesondere in Bereichen wie Bildsegmentierung, Objekterkennung und Klassifikation bahnbrechende Erfolge erzielt und gelten als Standardmethode in vielen modernen Anwendungen der Bildverarbeitung [Planche and Andres, 2019].

2.2.4.1. Convolution Layer

Die Convolution Layer (Faltungsschicht) bildet die wichtigste Komponente eines Convolutional Neural Networks (CNNs). Ihre Hauptaufgabe besteht darin, lokale Merkmale aus Eingabedaten, typischerweise Bildern, zu extrahieren. Dies geschieht mithilfe eines Filters (Kernels), der über das Eingangssignal geschoben („gefaltet“) wird, um sogenannte Feature Maps zu erzeugen.

Der mathematische Vorgang der Faltung eines Eingabebildes I mit einem Kernel K lässt sich folgendermaßen ausdrücken (2.14) [Planche and Andres, 2019]:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n) \tag{2.14}$$

Dabei (2.14) bezeichnet $S(i,j)$ den Wert an der Position (i,j) der Feature Map, I das Eingabebild und K den verwendeten Kernel.

Ein CNN verwendet meist mehrere unterschiedliche Kernels, wobei jeder Kernel spezifische Merkmale wie Kanten, Ecken oder Texturen erfasst. Diese Kernel werden mit einer definierten Schrittweite (Stride) (*Siehe Abbildung 2.5*) über die Eingabematrix verschoben. Der Stride bestimmt dabei, um wie viele Pixel der Kernel bei jedem Schritt horizontal oder vertikal weiterbewegt wird, und beeinflusst maßgeblich die räumliche Dimension der resultierenden Feature Maps. Die entstandenen Feature Maps dienen anschließend als Eingabe für weitere Schichten innerhalb des Netzwerks. Während des Trainingsprozesses werden die Gewichte der Kernel automatisch angepasst, sodass das CNN die relevanten Merkmale optimal identifizieren kann. Durch die Aneinanderreihung mehrerer Faltungsschichten ist das Netzwerk schließlich in der Lage, zunehmend

abstraktere und komplexere Merkmale zu erkennen, die für vielfältige Aufgaben der Mustererkennung entscheidend sind [Planche and Andres, 2019].

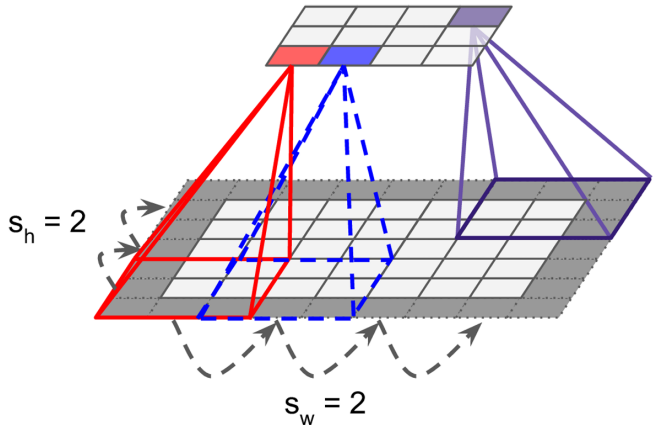


Abbildung 2.5.: Veranschaulichung der Convolution-Operation mit einer Schrittweite von 2 [Géron, 2022].

Die Verwendung mehrerer aufeinanderfolgender Faltungsschichten ermöglicht es CNNs, zunehmend abstraktere und komplexere Merkmale zu extrahieren, die für eine Vielzahl von Mustererkennungsaufgaben entscheidend sind [Planche and Andres, 2019].

2.2.4.2. Pooling Layer

Pooling-Schichten (*Pooling Layers*) werden in Convolutional Neural Networks verwendet, um die räumlichen Dimensionen der Merkmalskarten (*Feature Maps*) zu reduzieren. Dies verringert nicht nur den Rechenaufwand, sondern macht das Netzwerk auch robuster gegenüber kleinen Verschiebungen und Verzerrungen in den Eingabedaten.

Das meistgenutzte Pooling-Verfahren ist das sogenannte **Max-Pooling** (2.16). Bei diesem Verfahren wird ein festgelegter Bereich (*Pooling-Fenster*) betrachtet und aus diesem Bereich das Maximum als repräsentativer Wert ausgewählt. Mathematisch kann Max-Pooling wie folgt definiert werden (2.16) [Planche and Andres, 2019]:

$$P_{max}(R) = \max_{(i,j)\in R} S(i,j) \tag{2.16}$$

Dabei (2.16) steht $P_{max}(R)$ für den resultierenden Wert des Pooling-Schritts im Bereich R, während $S(i,j)$ den Wert der Feature Map an der Position (i,j) bezeichnet.

Alternativ wird gelegentlich das **Average-Pooling** (2.17) verwendet, das den Mittelwert aller Werte innerhalb des Fensters nimmt (2.17) [Planche and Andres, 2019]:

$$P_{avg}(R) = \frac{1}{|R|} \sum_{(i,j)\in R} S(i,j) \tag{2.17}$$

Dabei (2.17) bezeichnet $|R|$ die Anzahl der Elemente im Pooling-Fenster.

Durch Pooling-Schichten (*Siehe Abbildung 2.6*) können neuronale Netzwerke relevante Informationen kompakter und effizienter speichern, wodurch eine bessere Generalisierung und Robustheit erreicht wird.

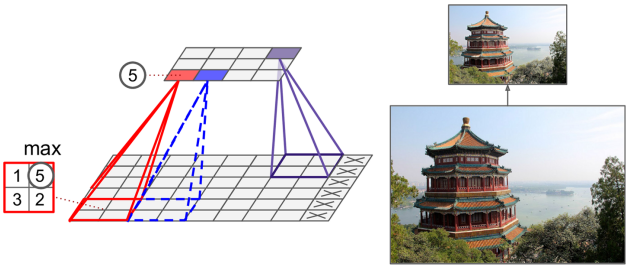


Abbildung 2.6.: Veranschaulichung von Max-Pooling [Géron, 2022].

Pooling-Schichten sind in modernen CNN-Architekturen unverzichtbar und tragen wesentlich zur hohen Leistungsfähigkeit bei Aufgaben wie Bildklassifikation und -segmentierung bei [Planche and Andres, 2019].

2.2.4.3. Aktivierungsfunktionen

Aktivierungsfunktionen verleihen neuronalen Netzwerken ihre Nichtlinearität, wodurch komplexe Muster effektiv erlernt werden können. Die Wahl der Aktivierungsfunktion beeinflusst dabei maßgeblich das Verhalten und die Lernfähigkeit des Netzwerks. Häufig verwendete Aktivierungsfunktionen sind:

Rectified Linear Unit (ReLU) ist derzeit die am häufigsten eingesetzte Aktivierungsfunktion in neuronalen Netzwerken, insbesondere in tiefen Netzwerken, da sie Berechnungen beschleunigt und das sogenannte „Vanishing Gradient"-Problem reduziert. Die mathematische Definition der ReLU-Funktion lautet (2.8) [Géron, 2022]:

$$f(z) = \max(0, z) \tag{2.8}$$

Sigmoid-Funktion wird typischerweise bei binären Klassifikationsaufgaben verwendet, da sie Ausgaben im Intervall zwischen 0 und 1 liefert und somit eine Wahrscheinlichkeit modellieren kann. Sie ist definiert als (2.9) [Géron, 2022]:

$$f(z) = \frac{1}{1 + e^{-z}} \tag{2.9}$$

Die **Softmax-Funktion** eignet sich für Klassifikationsaufgaben mit mehreren Klassen. Sie wandelt einen Vektor von beliebigen reellen Werten in eine Wahrscheinlichkeitsverteilung um, bei der die Werte im Intervall zwischen 0 und 1 liegen und deren Summe 1 ergibt. Die Funktion wird folgendermaßen definiert (2.10) [Géron, 2022]:

$$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \tag{2.10}$$

wobei z_i das i-te Element eines Vektors der Ausgabe-Schicht bezeichnet.

Die sorgfältige Auswahl der Aktivierungsfunktion ist entscheidend, da sie den Trainingsprozess, die Konvergenz und die Leistungsfähigkeit eines neuronalen Netzes direkt beeinflusst [Planche and Andres, 2019].

2.2.4.4. Regularisierung und Dropout

Regularisierungstechniken werden verwendet, um Überanpassung (*Overfitting*) neuronaler Netzwerke zu reduzieren und deren Generalisierungsfähigkeit zu verbessern. Eine häufig verwendete Methode zur Regularisierung ist die sogenannte L_2-Regularisierung (2.18), bei der die Verlustfunktion um einen zusätzlichen Term ergänzt wird, welcher die Gewichte bestraft (2.18) [Géron, 2022]:

$$L_{\text{reg}} = L + \lambda \sum_i w_i^2 \tag{2.18}$$

Hierbei (2.18) steht L für die ursprüngliche Verlustfunktion, w_i sind die Gewichte des neuronalen Netzwerks, und λ ist der Regularisierungsparameter, der die Stärke der Regularisierung bestimmt.

Eine weitere populäre Regularisierungstechnik ist **Dropout** (2.19) (*Siehe Abbildung 2.7*). Dropout reduziert Overfitting, indem während des Trainings zufällig ausgewählte Neuronen temporär deaktiviert werden. Der Output eines Neurons wird dabei mit einer Wahrscheinlichkeit p auf Null gesetzt. Formell lässt sich Dropout folgendermaßen beschreiben (2.19) [Géron, 2022]:

$$y_i^{(l)} = \begin{cases} 0 & \text{mit Wahrscheinlichkeit } p \\ \frac{y_i^{(l)}}{1-p} & \text{andernfalls} \end{cases} \tag{2.19}$$

wobei $y_i^{(l)}$ die Ausgabe des i-ten Neurons in Schicht l ist und p die Dropout-Rate bezeichnet. Der Faktor $\frac{1}{1-p}$ dient zur Skalierung, um sicherzustellen, dass der Erwartungswert unverändert bleibt.

Die richtige Anwendung von Regularisierungstechniken wie L_2-Regularisierung und Dropout ist entscheidend, um ein robustes neuronales Netz mit guter Generalisierungsfähigkeit zu erhalten [Planche and Andres, 2019].

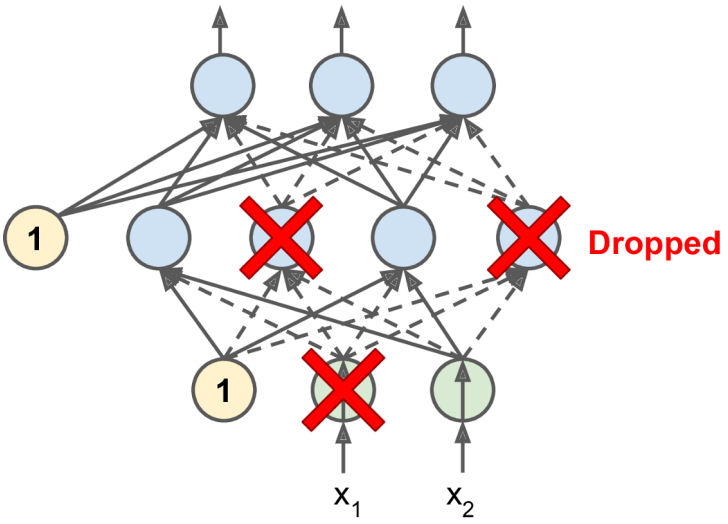


Abbildung 2.7.: Visualisierung der Dropout-Technik [Géron, 2022].

2.2.4.5. Überblick über relevante Architekturen

Im Bereich der Bildsegmentierung haben sich insbesondere Fully Convolutional Networks (FCN) und deren Weiterentwicklungen bewährt. Fully Convolutional Networks bestehen ausschließlich aus Faltungsschichten (*Convolutional Layers*) und ermöglichen somit eine pixelweise Klassifikation, wodurch sie ideal für Segmentierungsaufgaben sind.

Ein FCN verzichtet im Gegensatz zu traditionellen CNNs auf vollständig verbundene Schichten (*Fully Connected Layers*). Stattdessen erzeugt das Netzwerk aus einer Eingabe direkt eine Ausgabekarte (Segmentierungsmaske), welche die Klassifikation für jeden einzelnen Pixel repräsentiert.

Die Ausgabegröße eines FCN lässt sich dabei mathematisch ausdrücken als Funktion der Eingabegröße I, der Padding-Größe P, des verwendeten Filters F, des Strides S und der Anzahl der Schichten (2.20) [Planche and Andres, 2019]:

$$O = \frac{I - F + 2P}{S} + 1 \tag{2.20}$$

Ein FCN verwendet sogenannte Transposed Convolutions (auch Deconvolutions genannt), um die räumliche Dimension der Ausgabe wiederherzustellen und eine pixelweise Klassifikation zu ermöglichen (2.21) [Planche and Andres, 2019]:

$$O_{\text{transposed}} = (I - 1) \cdot S - 2P + F \tag{2.21}$$

Hierbei bezeichnet $O_{\text{transposed}}$ die Ausgabegröße nach einer Transposed Convolution.

Zu den bedeutenden Weiterentwicklungen des FCN gehört beispielsweise das **U-Net** (*Siehe Abbildung 2.8*) , welches zusätzlich sogenannte Skip-Verbindungen verwendet, um detaillierte Informationen direkt von frühen Schichten an spätere Schichten weiterzugeben. Dies verbessert insbesondere die Genauigkeit der Segmentierung [Planche and Andres, 2019].

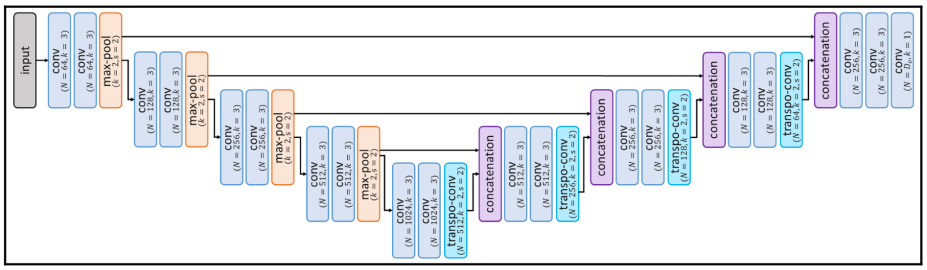


Abbildung 2.8.: Schematischer Aufbau des U-Net Architektur [Planche and Andres, 2019].

FCNs und deren Variationen stellen heute den Standard für viele Segmentierungsaufgaben dar, insbesondere im Bereich medizinischer Bildverarbeitung, und liefern hervorragende Ergebnisse bei hoher Effizienz.

2.3. Evaluationsmetriken für Segmentierungsaufgaben

In diesem Abschnitt werden die Evaluationsmetriken erläutert, die zur Beurteilung der Segmentierungsqualität verwendet wurden. Vorgestellt werden insbesondere der Dice Similarity Coefficient (DSC), Precision, Recall, der F1-Score sowie die Hintergrund-False-Positive-Rate. Abschließend wird die jeweilige Bedeutung dieser Metriken für die Interpretation und Bewertung der Ergebnisse diskutiert.

2.3.1. Dice Similarity Coefficient (DSC)

Der *Dice Similarity Coefficient* (DSC), auch bekannt als Sørensen–Dice-Koeffizient, ist eine häufig genutzte Evaluationsmetrik zur Beurteilung der Übereinstimmung zwischen einer vorhergesagten Segmentierung und einer Ground-Truth-Referenzmaske [Planche and Andres, 2019]. DSC misst, wie gut sich die vorhergesagte Maske mit der manuellen Annotation überlappt, und nimmt Werte zwischen 0 (keine Übereinstimmung) und 1 (perfekte Übereinstimmung) an.

Mathematisch definiert sich der DSC für zwei Mengen X (vorhergesagte Maske) und Y (Ground Truth) wie folgt (2.22)[Planche and Andres, 2019]:

$$DSC = \frac{2|X \cap Y|}{|X| + |Y|} \tag{2.22}$$

Hierbei bezeichnet $|X \cap Y|$ die Anzahl der Pixel, die sowohl in der vorhergesagten Maske als auch in der Ground Truth enthalten sind, während $|X|$ und $|Y|$ die Gesamtzahl der Pixel in der jeweiligen Segmentierung beschreiben. Ein hoher Dice-Koeffizient deutet auf eine hohe Segmentierungsgenauigkeit hin und ist somit entscheidend für die Bewertung medizinischer Segmentierungen [Planche and Andres, 2019].

2.3.2. Precision, Recall und F1-Score

Precision, Recall und F1-Score sind grundlegende Metriken zur Bewertung der Qualität einer Klassifikation oder Segmentierung, insbesondere im medizinischen Bereich. Sie basieren auf der Analyse von wahr positiven (True Positives, TP), falsch positiven (False Positives, FP) und falsch negativen (False Negatives, FN) Vorhersagen.

Precision (auch Präzision genannt) beschreibt den Anteil der tatsächlich relevanten Pixel unter allen als relevant vorhergesagten Pixeln. Precision ist definiert als (2.23) [Géron, 2022]:

$$\text{Precision} = \frac{TP}{TP + FP} \tag{2.23}$$

Recall (auch Sensitivität oder Trefferquote genannt) beschreibt den Anteil der korrekt vorhergesagten relevanten Pixel unter allen tatsächlich relevanten Pixeln. Recall berechnet sich wie folgt (2.24) [Géron, 2022]:

$$\text{Recall} = \frac{TP}{TP + FN} \tag{2.24}$$

Der **F1-Score** kombiniert Precision und Recall zu einer einzelnen, aussagekräftigen Metrik. Der F1-Score berechnet sich als harmonisches Mittel aus Precision und Recall und nimmt Werte zwischen 0 (schlechte Übereinstimmung) und 1 (perfekte Übereinstimmung) an (2.25) [Géron, 2022]:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{2.25}$$

Eine Segmentierung mit hohem Precision-Wert erzeugt wenige falsche Vorhersagen, während eine Segmentierung mit hohem Recall-Wert wenige relevante Pixel übersieht. Der F1-Score ist besonders nützlich, da er beide Aspekte ausgewogen berücksichtigt und somit einen umfassenden Eindruck über die Qualität der Segmentierung vermittelt [Géron, 2022].

3. Stand der Technik

Historisch gesehen wurde die Segmentierung von Zähnen zunächst durch klassische Methoden der Bildverarbeitung realisiert. Hierbei wurden Methoden wie Schwellenwert-basierte Verfahren, Regionenwachstum (region growing) sowie kanten- und konturbasierte Verfahren verwendet [Zhao et al., 2020]. Diese klassischen Ansätze haben jedoch einige grundlegende Limitationen: Schwellenwertverfahren sind empfindlich gegenüber Bildrauschen und Intensitätsinhomogenitäten, während konturbasierte Verfahren (z.B. Active Contours) anfällig für Initialisierungsfehler und lokale Minima sind [Çiçek et al., 2016].

Mit dem Aufkommen von tiefen neuronalen Netzen und insbesondere von Convolutional Neural Networks (CNNs) wurden erhebliche Fortschritte in der Segmentierungsgenauigkeit erzielt. Vor allem die U-Net Architektur und deren Derivate gelten heute als Goldstandard in der medizinischen Bildsegmentierung. Ursprünglich für die zweidimensionale Segmentierung mikroskopischer Bilder entwickelt, wurde das U-Net erfolgreich auf dreidimensionale Bilddaten übertragen (3D U-Net), was insbesondere bei komplexen medizinischen Datensätzen einen erheblichen Fortschritt darstellte [Çiçek et al., 2016].

[Çiçek et al., 2016] stellten eine Erweiterung der U-Net Architektur auf 3D-Bilder vor (3D U-Net), welche durch eine Encoder-Decoder-Struktur mit Skip-Verbindungen ausgezeichnet ist und eine effiziente Verarbeitung volumetrischer Daten erlaubt. Ein besonderes Merkmal der 3D U-Net Architektur ist, dass sie auch mit spars annotierten Datensätzen trainiert werden kann, wodurch der Aufwand manueller Segmentierung erheblich reduziert wird [Çiçek et al., 2016].

Eine weitere erfolgreiche Methode, welche speziell auf Röntgenbilder der Zähne abzielt, ist das Mask R-CNN Verfahren. Diese Methode kombiniert Objektdetektion und pixelgenaue Instanzsegmentierung und hat sich in komplexen Szenarien wie Panoramaröntgenbildern als robust erwiesen. Mask R-CNN liefert durch RoIAlign präzise segmentierte Masken und ist in der Lage, Zähne unabhängig voneinander genau abzugrenzen, was für die klinische Praxis besonders vorteilhaft ist [Jader et al., 2018].

Ein spezifischer Ansatz für die Segmentierung dentaler Panoramaröntgenbilder wurde durch [Zhao et al., 2020] vorgestellt. Der entwickelte „Two-Stage Attention Segmentation Network“ (TSASNet) integriert gezielt Aufmerksamkeitsmechanismen (Attention-

Mechanismen), um einerseits grob Zahnregionen zu lokalisieren und andererseits feine anatomische Details zu extrahieren. Diese Zwei-Stufen-Strategie verbessert signifikant die Präzision und die Robustheit gegenüber Herausforderungen wie niedrigem Kontrast und variierenden anatomischen Strukturen [Zhao et al., 2020].

Weitere Studien, wie etwa die Untersuchung von [Lee et al., 2018] zur Detektion von Karies mittels CNNs, demonstrieren die Effektivität tiefer Lernmethoden auch bei diagnostischen Fragestellungen in der Zahnmedizin. Durch eine GoogleNet-basierte CNN-Struktur wurde gezeigt, dass tief lernende Netzwerke in der Lage sind, Erkrankungen wie Karies zuverlässig und automatisiert aus dentalen Röntgenbildern zu diagnostizieren [Lee et al., 2018].

Ein zentrales Problem bleibt allerdings die generalisierbare Anwendung tiefer Lernmodelle aufgrund der oft begrenzten und heterogenen Datensätze in der zahnmedizinischen Bildgebung. [Silva et al., 2018] haben diesbezüglich ein umfangreiches Benchmark-Datensatz bereitgestellt, das nun eine einheitliche Grundlage für vergleichende Studien bietet und die Evaluierung neuer Algorithmen ermöglicht [Silva et al., 2018].

Zusammenfassend lässt sich festhalten, dass trotz erheblicher Fortschritte in der automatisierten dentalen Segmentierung durch tiefe neuronale Netze noch Herausforderungen bestehen. Insbesondere in der präzisen Abgrenzung feinster anatomischer Strukturen, der Robustheit gegenüber Bildartefakten und der Generalisierbarkeit über unterschiedliche Bildmodalitäten hinweg besteht weiterer Forschungsbedarf. Die Integration von aufmerksamkeit-basierten Methoden, 3D-U-Netzwerken sowie Ansätzen wie Mask R-CNN zeigt jedoch vielversprechende Potenziale für zukünftige Entwicklungen in diesem Bereich.

4. Methoden und Ergebnisse

In diesem Kapitel werden die eingesetzten Methoden und der experimentelle Aufbau der Arbeit ausführlich vorgestellt. Dies beinhaltet eine detaillierte Beschreibung der verwendeten Datensätze und deren Vorverarbeitung, die eingesetzten Deep-Learning-Modelle, die Trainings- und Optimierungsprozesse sowie die verwendeten Evaluationsmetriken. Abschließend werden die erzielten Ergebnisse quantitativ und qualitativ dargestellt, analysiert und interpretiert.

4.1. Experimenteller Aufbau und Datensatz

In diesem Abschnitt werden der verwendete Datensatz sowie der experimentelle Aufbau detailliert erläutert. Dazu gehört eine Beschreibung der Datensatzmerkmale, auftretender Probleme und notwendiger Änderungen bei der Datensatzerstellung sowie der Vorverarbeitungsschritte, die zur Vorbereitung der Daten auf das Modelltraining durchgeführt wurden.

4.1.1. Experimentumgebung

Um die Experimente dieser Arbeit durchzuführen und reproduzierbare Ergebnisse sicherzustellen, wurden sämtliche Trainings- und Testprozesse auf einer speziell dafür vorgesehenen Hardware- und Softwareumgebung ausgeführt. Alle Trainings- und Testprozesse wurden unter Verwendung einer GPU beschleunigt ausgeführt. Die untenstehenden Tabellen geben detaillierte Auskunft über die verwendeten Softwarebibliotheken sowie die technischen Spezifikationen des Computersystems.

Tabelle 4.1.: Verwendete Softwarebibliotheken und Versionen

Bibliothek	**Version**
Albumentations	2.0.6
Matplotlib	3.10.3
NumPy	1.26.4
OpenCV (headless)	4.11.0.86
Pillow	11.0.0
Pycocotools	2.0.8
Scikit-image	0.25.2
SciPy	1.15.3
Segmentation_models_pytorch	0.5.0
Tensorboard	2.19.0
Timm	1.0.15
Torch	2.0.1+cu117
Torchvision	0.15.2+cu117
Tqdm	4.67.1

Tabelle 4.2.: Technische Spezifikationen des Computersystems

Komponente	**Spezifikation**
Prozessor (CPU)	AMD Ryzen 5 7600, 6-Core, 3801 MHz, 12 Threads
Arbeitsspeicher (RAM)	32 GB DDR5, 3000 MHz
Betriebssystem (OS)	Windows 10, 64-Bit
Festplatte (SSD)	Corsair MP600 CORE XT, 2 TB
Grafikkarte (GPU)	NVIDIA GeForce RTX 4060 Ti, 16 GB GDDR6

4.1.2. Datensatz

Für diese Arbeit wurde den Datensatz „ToothFairy2“ verwendet, der im Rahmen der ToothFairy 2 Challenge der Konferenz MICCAI2024 entstanden ist. Diese Challenge wurde in Zusammenarbeit zwischen der University of Modena and Reggio Emilia und der Radboud University organisiert. Der ToothFairy2-Datensatz stellt eine erweiterte und verbesserte Version der zuvor veröffentlichten Datensätze „Maxillo“ und „ToothFairy“ dar [Lumetti et al., 2024].

Der ToothFairy2-Datensatz besteht aus zwei Teilmengen:

- **Set A:** Diese Teilmenge umfasst 417 Volumendatensätze, die teilweise mit dem bereits veröffentlichten ToothFairy-Datensatz überlappen. Die Dateinamen dieser Datensätze beginnen mit dem Buchstaben „P“.
- **Set B:** Diese Teilmenge umfasst 63 Volumendatensätze, die neu sind und bislang weder im ToothFairy- noch im Maxillo-Datensatz veröffentlicht wurden. Die Dateinamen dieser Datensätze beginnen mit dem Buchstaben „F“. Diese Datensätze bieten ein größeres Sichtfeld, insbesondere vollständige Segmentierungen der oberen Zähne.

Die Daten liegen im „mha“-Format vor und sind in der Hounsfield-Skala codiert. Insgesamt umfasst der Datensatz 42 Klassen. Dazu gehören der Hintergrund, Kieferknochen, die unteren Alveolarkanäle, Kieferhöhlen, Pharynx, Brücken, Kronen, Implantate sowie obere und untere Zähne (einschließlich der Weisheitszähne). Das größte Volumen (F039) hat die Dimensionen (298, 512, 512), das kleinste Volumen (P381) misst (170, 272, 345), und die mittlere Volumengröße (P460) liegt bei (170, 357, 371). Die Struktur des Datensatzes entspricht dem nnU-Net-Format, bestehend aus Rohbildern, zugehörigen Segmentierungslabels und einer Metadatendatei („dataset.json“) [Lumetti et al., 2024].

4.1.2.1. Probleme beim ursprünglichen 3D-Datensatz

Während der Bearbeitung des ursprünglich geplanten 3D-Datensatzes sind diverse technische Schwierigkeiten aufgetreten. Die Verarbeitung der hochdimensionalen 3D-Daten erforderte erhebliche Rechenleistung und Speicherressourcen, die mit der verfügbaren Hardware-Infrastruktur nicht vollständig kompatibel waren. Dies führte zu erheblichen Verzögerungen während der Trainingsphase und sogar zu vollständigen Unterbrechungen des Trainingsprozesses. Somit geriet der vorgesehene Zeitplan für die Projektarbeit deutlich in Gefahr. Darüber hinaus enthielten nur wenige Datensätze Tomographiebilder des Unterkiefers, wodurch die Segmentierung auf Zähne im Oberkiefer beschränkt werden musste.

4.1.2.2. Änderungen am verwendeten Datensatz

Um die genannten technischen Herausforderungen zu überwinden, wurde eine wesentliche Anpassung am Datensatz vorgenommen. Anstelle der vollständigen 3D-Volumina wurden spezifische zweidimensionale Schnitte (Slices) aus den tomographischen Bildern der Patienten extrahiert und damit ein neuer, handhabbarer 2D-Datensatz erstellt. Die ausgewählten Schnitte wurden anhand optimaler anatomischer Sichtbarkeit der relevanten Strukturen ausgewählt, um die für die Segmentierung benötigten grundlegenden Informationen zu bewahren.

4.1.2.3. Eigenschaften und Vorverarbeitung des neuen 2D-Datensatzes

Der neu erstellte 2D-Datensatz fokussiert sich ausschließlich auf Bilder der Oberkieferzähne. Jedes Bild im Datensatz enthält den Hintergrund sowie die Zielklassen der oberen Zähne, die gemäß der FDI-Klassifikation mit den Codes 11 bis 18 und 21 bis 28 versehen sind. Bereiche, die nicht zur Segmentierung relevant sind, wurden als „irrelevante Region“ markiert und mit einem speziellen Wert versehen. Die Bilddaten liegen im „numpy“-Format vor und sind jeweils mit entsprechenden Masken gekoppelt. Während der Vorverarbeitung wurden die Pixelwerte der Bilder auf den Wertebereich $[0, 1]$ normalisiert. Die Segmentierungsmasken wurden von ihren ursprünglichen Klassenwerten (FDI-Codes) auf eine vereinfachte Klassenstruktur umkodiert, bestehend aus den Zielzähnen (1–16), dem Hintergrund (0) sowie irrelevanten Bereichen. Der Datensatz wurde in drei Teilmengen unterteilt: Training (80%), Validierung (10%) und Test (10%). Der Trainingsdatensatz dient der Modellierung und dem Training der Modelle, während der Validierungsdatensatz zur Überwachung und Optimierung der Modellleistung während des Trainings dient. Der Testdatensatz wurde schließlich verwendet, um nach Abschluss des Trainings die Generalisierungsfähigkeit der Modelle zu bewerten. Durch diese neue Datensatzstruktur konnte eine recheneffiziente und klinisch praktikable Grundlage geschaffen werden, um Deep-Learning-Modelle erfolgreich und ressourcenschonend zu trainieren.

4.2. Verwendete Methoden

In diesem Abschnitt werden die im Rahmen dieser Arbeit eingesetzten Deep-Learning-Modelle detailliert beschrieben. Dazu gehören die grundlegenden Eigenschaften und Besonderheiten der Modelle, ihre jeweilige Netzwerkarchitektur sowie die Gründe für deren Auswahl im Hinblick auf die Segmentierungsaufgabe.

4.2.1. Verwendete Deep-Learning-Modelle

In diesem Abschnitt werden die spezifischen Deep-Learning-Modelle vorgestellt, die für die Segmentierungsaufgabe verwendet wurden. Neben einer Beschreibung ihrer jeweiligen Architekturen werden die charakteristischen Merkmale und Besonderheiten hervorgehoben, welche die Modelle besonders geeignet für die gestellte Aufgabe machen.

4.2.1.1. UNet

Das UNet-Modell ist eine leistungsfähige und weit verbreitete Deep-Learning-Architektur, die speziell für die biomedizinische Bildsegmentierung entwickelt wurde. Dieses Netzwerk wurde von Olaf Ronneberger und Kollegen vorgestellt und basiert im Wesentlichen auf der Idee der vollständig faltenden Netzwerke (Fully Convolutional Networks, FCNs). UNet zielt darauf ab, bereits mit einer begrenzten Anzahl von Trainingsdaten qualitativ hochwertige Segmentierungsergebnisse zu erzielen. Die Architektur von UNet besteht aus zwei wesentlichen Teilen: einem sogenannten Encoder-Pfad (Verkleinerungspfad) und einem Decoder-Pfad (Vergrößerungspfad) [Ronneberger et al., 2015].

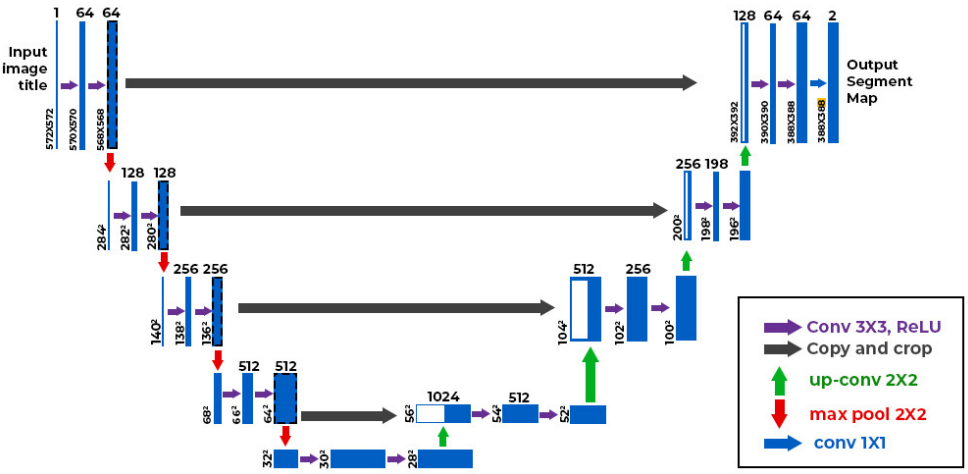


Abbildung 4.1.: Architektur des UNet-Modells [Ronneberger et al., 2015].

Der Encoder-Pfad folgt einer typischen CNN-Architektur und ist dafür zuständig, aus dem Eingabebild Kontextinformationen zu extrahieren. In diesem Teil des Netzwerks werden Bildmerkmale mithilfe aufeinanderfolgender 3x3-Faltungsschichten extrahiert, wobei nach jeder Faltung eine nicht-lineare Aktivierung mittels ReLU (Rectified Linear Unit) angewendet wird. Anschließend wird die räumliche Dimension der Merkmalskarten durch 2x2 Max-Pooling-Operationen reduziert. Mit jedem Pooling-Schritt verdoppelt sich dabei die Anzahl der Kanäle, was zu tiefgehenden und abstrakten Merkmalrepräsentationen führt [Ronneberger et al., 2015].

Der Decoder-Pfad ist so konzipiert, dass er aus den zuvor verkleinerten Merkmalskarten hochauflösende Segmentierungsergebnisse generiert. Zunächst werden die reduzierten Merkmalskarten mithilfe sogenannter „Up-Convolution"-Operationen (2x2-Faltung für das Upsampling) schrittweise wieder vergrößert. Gleichzeitig erfolgt die Integration sogenannter Skip-Verbindungen (skip connections), die detaillierte Informationen aus den entsprechenden Encoder-Schichten direkt in den Decoder übertragen. Dadurch bleiben feine Strukturen und wichtige anatomische Details erhalten. Zum Abschluss werden zwei weitere 3x3-Faltungsschichten mit ReLU-Aktivierungen angewandt, um die Merkmalsrepräsentation zusätzlich zu verbessern. Insgesamt umfasst die UNet-Architektur 23 Faltungsschichten [Ronneberger et al., 2015].

Das UNet nutzt zudem die sogenannte „Overlap-Tile"-Strategie, die es ermöglicht, auch großformatige Bilder ohne Verlust zu segmentieren. Bei dieser Methode wird fehlende Kontextinformation an den Bildrändern durch Spiegelung ergänzt, wodurch präzisere und qualitativ hochwertigere Segmentierungen erzielt werden können [Ronneberger et al., 2015].

Während des Trainings von UNet wird in der Regel eine Kombination aus der Kreuzentropie-Verlustfunktion (Cross-Entropy Loss) und einer umfangreichen Datenaugmentation angewandt. Diese Datenaugmentation beinhaltet vor allem elastische Deformationen, die dabei helfen, die Generalisierungsfähigkeit des Netzwerks deutlich zu verbessern, sodass auch mit einer begrenzten Anzahl an Trainingsdaten hohe Segmentierungsgenauigkeiten erzielt werden können [Ronneberger et al., 2015].

Das UNet-Modell eignet sich besonders gut für die Zahnsegmentierung, da es in der Lage ist, kleine anatomische Details präzise zu erfassen. Die Verwendung von Skip-Verbindungen stellt sicher, dass feine Strukturen der Zähne bewahrt werden, während der Decoder-Pfad scharfe und präzise Segmentierungsgrenzen liefert. Zudem erhöht die Integration elastischer Deformationen in der Datenaugmentation die Robustheit des Modells gegenüber variierenden Bilddaten, was es für klinische Anwendungen zur Zahnsegmentierung äußerst effektiv und zuverlässig macht [Ronneberger et al., 2015].

4.2.1.2. BiSeNetV2

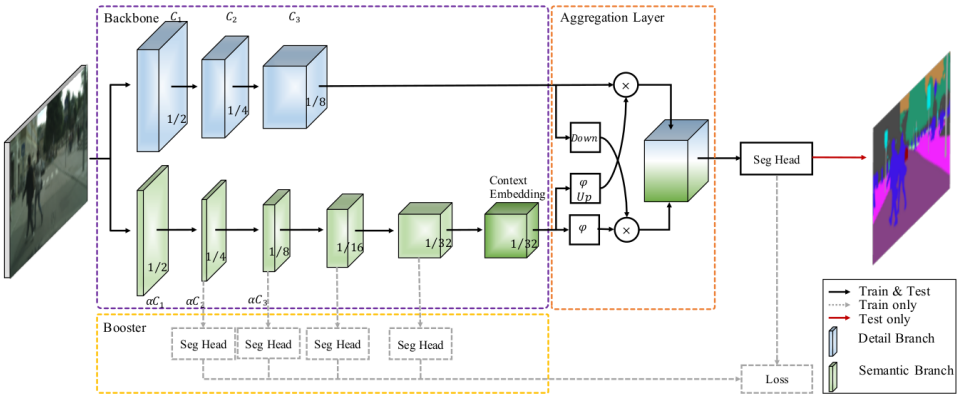


Abbildung 4.2.: Architektur des BiSeNetV2-Modells [Yu et al., 2021].

BiSeNetV2 ist eine schnelle und effiziente Dual-Path-Deep-Learning-Architektur, die speziell für semantische Segmentierungsaufgaben in Echtzeit entwickelt wurde. Das Modell verarbeitet niedrigstufige Details und höherstufige semantische Informationen in zwei separaten Pfaden: einem Detail-Pfad (*Detail Branch*) und einem semantischen Pfad (*Semantic Branch*). Der Detail-Pfad besteht aus flachen, aber breiten Schichten, welche besonders die feineren, hochauflösenden Details eines Bildes erfassen. Der semantische Pfad hingegen nutzt tiefere Schichten, eine geringere Kanalzahl sowie eine schnelle Downsampling-Strategie, um umfassende, semantische Kontexte zu extrahieren [Yu et al., 2021].

Um die Information aus diesen beiden Pfaden effektiv zu integrieren, verwendet BiSeNetV2 eine sogenannte *Bilateral Guided Aggregation Layer* (bilateral geführte Aggregationsschicht). Diese Schicht nutzt hochstufige, semantische Informationen aus dem semantischen Pfad, um niedrigstufige, detaillierte Merkmale aus dem Detail-Pfad anzureichern und zu leiten. Dadurch wird ein umfassender, multi-skaliger Kontext geschaffen, der sowohl grobe semantische Zuordnungen als auch feine Details effektiv abbilden kann [Yu et al., 2021].

Ein zentrales Merkmal von BiSeNetV2 ist dessen schnelle Inferenzgeschwindigkeit, die entscheidend für Echtzeitanwendungen ist. Diese Geschwindigkeit wird durch eine Kombination aus reduzierten Kanalkapazitäten und einer effizienten Architektur erreicht. Besonders hervorzuheben ist die sogenannte *Gather-and-Expansion Layer* (Sammel- und Erweiterungsschicht). Diese ähnelt der *Inverted-Bottleneck*-Struktur, wie sie häufig in modernen Deep-Learning-Netzen verwendet wird. Hierbei wird durch den Einsatz tiefenbasierter, effizienter Faltungen der rezeptive Bereich des Netzwerks erweitert und gleichzeitig die Berechnungslast reduziert [Yu et al., 2021].

Aufgrund seiner Fähigkeit, sowohl feine Details als auch umfassende semantische Informationen getrennt zu verarbeiten und anschließend effektiv zu integrieren, eignet sich BiSeNetV2 besonders gut für die Segmentierung von Zähnen. Die Architektur ermöglicht es, selbst kleine anatomische Strukturen und deren genaue Grenzen zuverlässig zu erfassen. Zudem bietet die schnelle Verarbeitungsgeschwindigkeit von BiSeNetV2 entscheidende Vorteile für klinische Anwendungen, bei denen Echtzeitsegmentierungen oft von großer Bedeutung sind [Yu et al., 2021].

4.2.1.3. DenseASPP

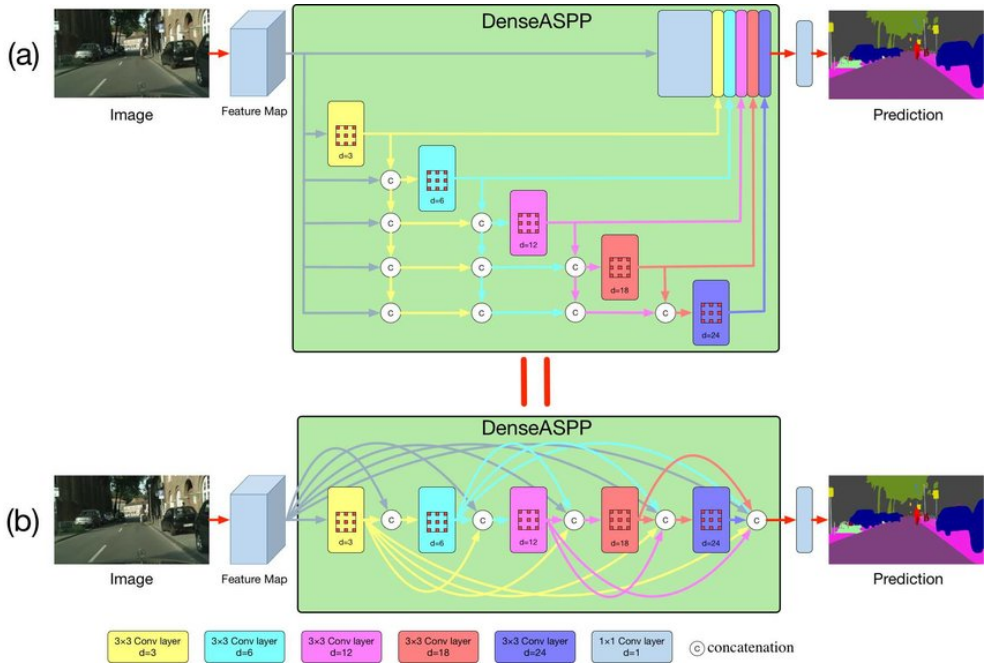


Abbildung 4.3.: Architektur des DenseASPP-Modells [Yang et al., 2018].

Das DenseASPP-Modell stellt eine innovative Deep-Learning-Architektur dar, die speziell dafür entwickelt wurde, multiskalare Kontextinformationen zu erfassen und gleichzeitig ein dichtes und breites rezeptives Feld zu schaffen. Die Architektur baut auf dem Konzept des *Atrous Spatial Pyramid Pooling (ASPP)* auf, wobei Dilated-Convolution-Schichten (Atrous-Convolution) mit verschiedenen Dilationsraten verwendet und diese über sogenannte dichte Verbindungen (*dense connections*) eng miteinander verknüpft werden. Durch diesen Ansatz kombiniert das DenseASPP-Modell Merkmale unterschiedlicher Skalen äußerst intensiv und konstruiert dadurch eine umfassende und dichte Merkmals-Pyramide [Yang et al., 2018].

Innerhalb der DenseASPP-Architektur werden die Ausgaben jeder Dilated-Convolution-Schicht nicht isoliert betrachtet, sondern jeweils mit den Ausgaben aller vorangegan-

genen Schichten kombiniert und an die folgenden Schichten weitergegeben. Dieser intensive und dichte Datenaustausch zwischen den Schichten ermöglicht es, Merkmalskarten zu generieren, die eine große Bandbreite an Details und Kontextinformationen auf mehreren Skalen abdecken. Durch diesen Ansatz werden Objekte oder Strukturen auf unterschiedlichen Maßstäben wesentlich detaillierter und präziser dargestellt [Yang et al., 2018].

Dank dieser dichten Vernetzung und intensiven Merkmalsintegration kann das Modell bei jedem Verarbeitungsschritt eine umfangreichere und detailliertere Abtastung des Eingabebildes durchführen. Dies stellt insbesondere bei Segmentierungsaufgaben, wie etwa der Zahnsegmentierung, einen entscheidenden Vorteil dar. Gerade bei kleinen und komplexen anatomischen Strukturen ermöglicht diese Vorgehensweise, feine Details sowie Grenzen zwischen den einzelnen Zähnen äußerst präzise und zuverlässig zu erkennen. So werden durch das DenseASPP-Modell deutlich schärfere und genauere Segmentierungsergebnisse erzielt [Yang et al., 2018].

Zusammenfassend lässt sich sagen, dass das DenseASPP-Modell aufgrund seiner Fähigkeit, intensive und umfassende multiskalare Kontextinformationen sowie ein breites rezeptives Feld zu generieren, besonders gut für Anwendungen geeignet ist, bei denen feine anatomische Details, wie in der Zahnsegmentierung, von entscheidender Bedeutung sind [Yang et al., 2018].

4.2.1.4. SegNet

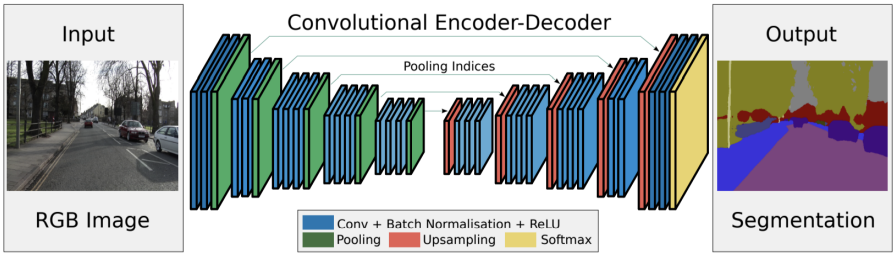


Abbildung 4.4.: Architektur des SegNet-Modells [Badrinarayanan et al., 2017].

SegNet ist eine effiziente Deep-Learning-Architektur für die Bildsegmentierung, basierend auf einer Encoder-Decoder-Struktur. Das von Vijay Badrinarayanan und Kollegen entwickelte Modell wurde speziell für semantische Segmentierungsaufgaben konzipiert und zeichnet sich durch architektonische Einfachheit sowie eine hohe Speichereffizienz aus. Die SegNet-Architektur orientiert sich maßgeblich am tiefen CNN-Modell VGG16 und interpretiert dessen Struktur für Segmentierungsaufgaben neu [Badrinarayanan et al., 2017].

Die SegNet-Architektur besteht im Wesentlichen aus zwei Hauptteilen: dem Encoder (Kodierer) und dem Decoder (Dekodierer). Der Encoder-Teil ist darauf ausgelegt, aus dem Eingangsbild tiefgehende und aussagekräftige Merkmale zu extrahieren. In dieser Phase werden mehrere aufeinanderfolgende Faltungsschichten verwendet, um die Bildinformationen in kompaktere, aber semantisch reichhaltigere Merkmalsdarstellungen zu transformieren. Jede Faltungsschicht wird durch eine nichtlineare Aktivierungsfunktion (ReLU) gefolgt. Danach reduziert eine Max-Pooling-Schicht die räumliche Dimension der Merkmalskarten. Während des Max-Pooling-Prozesses werden die Positionen der maximalen Aktivierungswerte (Pooling-Indizes) in jedem räumlichen Bereich gespeichert. Diese gespeicherten Indizes werden später im Decoder verwendet und stellen somit ein wesentliches Alleinstellungsmerkmal von SegNet gegenüber anderen Segmentierungsmodellen dar [Badrinarayanan et al., 2017].

Der Decoder-Teil rekonstruiert die zuvor verkleinerten Merkmalskarten wieder in hoher Auflösung. Ein wesentlicher innovativer Ansatz von SegNet besteht darin, die im Encoder gespeicherten Max-Pooling-Indizes direkt für das Upsampling im Decoder zu verwenden. Durch diese Indizes ist der Decoder exakt darüber informiert, welche räumlichen Positionen während des Upsampling-Schritts aktiviert werden sollen. Dies ermöglicht im Vergleich zu traditionellen Ansätzen eine wesentlich präzisere und schärfere Segmentierungsgrenze. Nach dem Upsampling-Prozess werden mehrere wei-

tere Faltungsschichten verwendet, um hochauflösende, detaillierte Segmentierungsmasken zu generieren [Badrinarayanan et al., 2017].

Die Verwendung der Max-Pooling-Indizes ermöglicht es SegNet, mit begrenzten Speicherressourcen äußerst effizient zu arbeiten. Da die Indizes direkt räumliche Informationen ohne zusätzliche lernbare Parameter bereitstellen, reduziert dies den allgemeinen Speicherverbrauch des Modells erheblich. Dies macht SegNet insbesondere für Anwendungen attraktiv, die schnelle Verarbeitung erfordern und nur begrenzte Hardware-Ressourcen zur Verfügung haben, wie es häufig in klinischen Umgebungen der Fall ist [Badrinarayanan et al., 2017].

Für die Zahnsegmentierung ist SegNet besonders gut geeignet, da es in der Lage ist, Segmentierungsmasken in hoher Auflösung mit scharfen und präzisen Grenzen zu erzeugen. Die anatomischen Strukturen der Zähne liegen oftmals sehr nah beieinander und weisen häufig unscharfe Übergänge auf, was die Segmentierungsaufgabe herausfordernd macht. Durch die Encoder-Decoder-Struktur und insbesondere die Nutzung der gespeicherten Pooling-Indizes kann SegNet feinste Details und genaue anatomische Grenzen zuverlässig erfassen. Zusätzlich macht die effiziente Speichernutzung das Modell besonders attraktiv für Echtzeitanwendungen in klinischen Umgebungen. Aufgrund dieser Stärken eignet sich SegNet hervorragend für Aufgaben wie die Zahnsegmentierung, bei denen eine genaue und detaillierte Segmentierung unerlässlich ist [Badrinarayanan et al., 2017].

4.2.1.5. BASNet

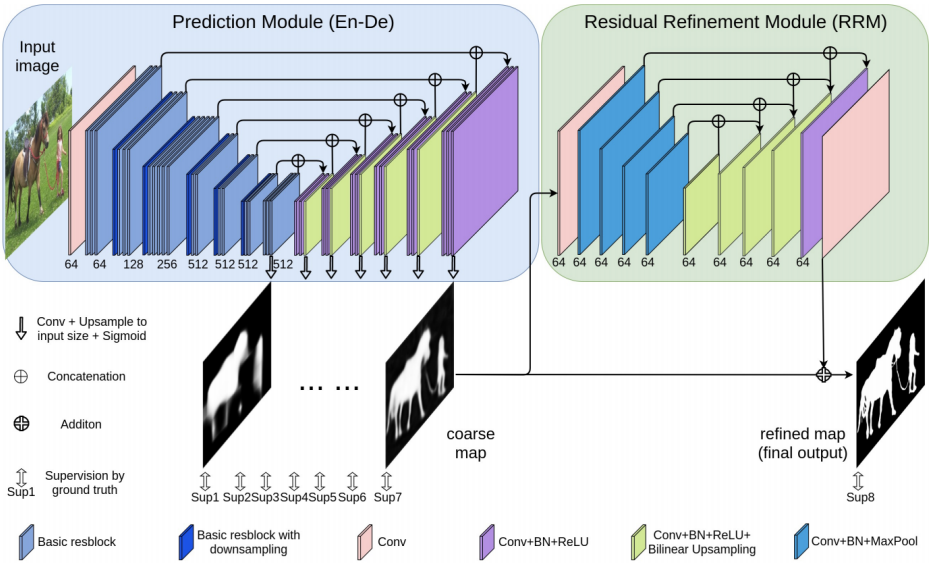


Abbildung 4.5.: Architektur des BASNet-Modells [Qin et al., 2021].

Das *Boundary-Aware Segmentation Network* (BASNet) ist ein Deep-Learning-basiertes Bildsegmentierungsmodell, das speziell zur präzisen und exakten Bestimmung der Grenzen von Objekten entwickelt wurde. Das von Qin und Kollegen vorgeschlagene Modell zielt darauf ab, besonders die Grenz- und Randdetails der zu segmentierenden Objekte genau und scharf zu bestimmen. BASNet verfolgt dazu eine zweistufige Strategie, die unter dem Namen „Predict-Refine“ (Vorhersage-Verfeinerung) bekannt ist und dazu dient, die Qualität der generierten Segmentierungsmasken stufenweise zu verbessern [Qin et al., 2021].

In der ersten Phase, der sogenannten „Predict“-Phase (Vorhersagephase), werden erste Segmentierungsergebnisse durch eine Encoder-Decoder-Struktur erzeugt, die auf dem Prinzip der tiefen Überwachung (*deep supervision*) basiert. Dabei werden tiefe Merkmalskarten aus den Bildern extrahiert, welche die Segmentierungsmasken auf unterschiedlichen Skalen generieren. Durch diesen Ansatz der tiefen Überwachung lernt das Netzwerk, bereits während des Trainings mehrere Auflösungen von Segmentierungsmasken gleichzeitig und effektiv vorherzusagen. Diese Herangehensweise verbessert maßgeblich die Generalisierungsfähigkeit und damit die Gesamtleistung des Modells. Im Encoder kommen ResNet-ähnliche Faltungsschichten zum Einsatz, um tiefe semantische Merkmale aus den Eingabebildern zu extrahieren. Der Decoder wiederum stellt die räumliche Auflösung mithilfe von Skip-Verbindungen und Upsampling-Operationen wieder her und erzeugt dadurch präzise Segmentierungsmasken [Qin et al., 2021].

Die zweite Phase von BASNet, das sogenannte „Refine“-Modul (Verfeinerungsmodul), zielt darauf ab, die Ergebnisse der ersten Vorhersagephase weiter zu verbessern und zu verfeinern. Dieses Modul konzentriert sich darauf, die Differenz zwischen den anfänglich generierten Segmentierungsmasken und den tatsächlichen (*ground truth*) Masken zu minimieren. Dazu verwendet das Refine-Modul einen residualen (Restfehler-) Lernansatz, mit dessen Hilfe insbesondere Grenzbereiche verfeinert und präzisiert werden. Durch diesen Ansatz werden die Grenzen der segmentierten Objekte besonders scharf und präzise. Das Modell erhöht somit seine Fähigkeit zur Grenzerkennung (*boundary-awareness*), was wiederum die Qualität der Segmentierung erheblich verbessert, insbesondere bei komplexen und detailreichen Strukturen [Qin et al., 2021].

BASNet ist besonders gut für Anwendungen wie die Zahnsegmentierung geeignet, da das Modell über eine sehr hohe Präzision bei der Erkennung von Grenzregionen verfügt. Zähne weisen anatomisch bedingt häufig sehr enge und schwer voneinander abgrenzbare Strukturen auf, was die Segmentierungsaufgabe besonders anspruchsvoll macht. Dank des zweistufigen Vorhersage- und Verfeinerungsverfahrens von BASNet können die komplexen und feinen Grenzstrukturen der Zähne mit hoher Genauigkeit bestimmt werden. Zudem erlaubt der Ansatz der tiefen Überwachung eine effektive Erkennung

struktureller Details auf unterschiedlichen Skalen, wodurch die Leistungsfähigkeit des Modells zusätzlich erhöht wird. Aufgrund dieser Eigenschaften erweist sich BASNet besonders für klinische Anwendungen als wertvolles Instrument, wo eine hochpräzise und zuverlässige Segmentierung feiner anatomischer Strukturen wie der Zähne unabdingbar ist [Qin et al., 2021].

4.2.1.6. Mask R-CNN

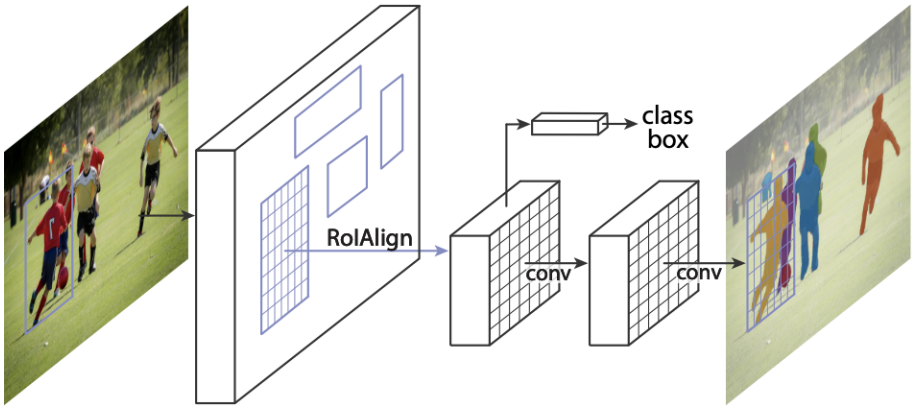


Abbildung 4.6.: Architektur des Mask R-CNN-Modells [He et al., 2017].

Mask R-CNN (Mask Region-Based Convolutional Neural Network) stellt ein leistungsstarkes Deep-Learning-Verfahren dar, welches Methoden der Objekterkennung und der Instanzsegmentierung in einem einzigen Modell kombiniert. Entwickelt wurde dieses Modell von Kaiming He und Kollegen auf Grundlage der bewährten Faster R-CNN Architektur. Im Unterschied zu klassischen Objekterkennungsverfahren liefert Mask R-CNN neben der üblichen Objektklassifikation und Bounding-Box-Lokalisierung zusätzlich präzise, pixelgenaue Segmentierungsmasken für jedes detektierte Objekt. Dadurch wird eine detaillierte und exakte Abgrenzung der Objektkonturen ermöglicht [He et al., 2017].

Die Struktur des Mask R-CNN-Modells besteht im Wesentlichen aus zwei Phasen. Zunächst werden potenzielle Objektbereiche im Bild mittels eines Region Proposal Networks (RPN) identifiziert. Das RPN schlägt Bounding Boxes vor, welche Bereiche markieren, in denen sich relevante Objekte befinden könnten. Anschließend werden diese Vorschläge in einer zweiten Phase in einer parallelen Struktur verarbeitet, welche aus zwei unabhängigen Zweigen besteht. Der erste Zweig übernimmt hierbei die Klassifikation und Feinlokalisierung der Objekte durch Bounding-Box-Regression, während

der zweite Zweig gleichzeitig eigenständige, pixelgenaue Segmentierungsmasken generiert [He et al., 2017].

Eine wesentliche Weiterentwicklung des Mask R-CNN ist die Einführung des RoIAlign-Verfahrens, das speziell entwickelt wurde, um räumliche Ungenauigkeiten (Spatial Misalignment), die im Faster R-CNN durch den RoIPool-Algorithmus entstehen, zu vermeiden. RoIPool arbeitet mit einer festen Unterteilung der Merkmalskarten und Max-Pooling, was jedoch aufgrund der dabei entstehenden Quantisierung zu Genauigkeitsverlusten führen kann. Im Gegensatz dazu verwendet RoIAlign kontinuierliche Werte und berechnet Merkmale mithilfe bilinearer Interpolation direkt und präzise, sodass selbst feine und detaillierte Strukturen exakt dargestellt werden können [He et al., 2017].

Der Maskenzweig des Mask R-CNN-Modells ist als eigenständiges, vollständig aus Faltungsschichten bestehendes Netzwerk aufgebaut, das eigenständig für jede vorgeschlagene Region pixelweise Masken erzeugt. Dabei werden für jede Objektklasse unabhängige binäre Masken erstellt. Diese unabhängige und parallele Arbeitsweise stellt sicher, dass die Segmentierungsmasken und Klassifikationen sich gegenseitig nicht beeinflussen, was den Trainingsprozess optimiert und die Präzision der generierten Masken erhöht [He et al., 2017].

Mask R-CNN eignet sich besonders gut für die Zahnsegmentierung, da es in der Lage ist, detailreiche und hochpräzise Segmentierungen zu liefern und kleine anatomische Strukturen präzise zu erfassen. Insbesondere im klinischen Kontext der Zahnmedizin, wo klare und genaue Grenzen entscheidend sind, bietet das RoIAlign-Verfahren eine zuverlässige und exakte Erkennung feinster anatomischer Details. Durch die unabhängige Segmentierung jedes Zahnes erzielt Mask R-CNN auch bei variablen anatomischen Formen und Größen hervorragende Ergebnisse, wodurch es sich als klinisch hoch relevant und besonders effektiv für die Zahnsegmentierung erweist [He et al., 2017].

4.2.2. Optimierung der Modelle

4.2.2.1. Verlustfunktion

In dieser Arbeit wurden beide Verlustfunktionen mit identischer Gewichtung ($\alpha = 1$, $\beta = 1$) verwendet, um ein ausgewogenes Gleichgewicht zwischen pixelbasierter Präzision (Cross-Entropy) und struktureller Kongruenz (Dice Loss) zu gewährleisten.

Dice Loss

Der verwendete Dice Loss ist eine mehrklassige Verlustfunktion, welche die Überlappung zwischen der vorhergesagten Segmentierungsmaske und der tatsächlichen Maske misst.

Dabei wird ein sogenannter *ignore_index* unterstützt, wodurch bestimmte Klassen, beispielsweise der Hintergrund oder irrelevante Bereiche, bei der Berechnung ausgeschlossen werden können. Der Dice-Koeffizient für eine Klasse c ist definiert als (4.1) [Planche and Andres, 2019]:

$$Dice_c = \frac{2 \sum_{i=1}^{N} p_{ci} \cdot t_{ci} + \epsilon}{\sum_{i=1}^{N} (p_{ci} + t_{ci}) + \epsilon} \tag{4.1}$$

Hierbei sind:

- p_{ci}: vorhergesagte Wahrscheinlichkeit, dass Pixel i zur Klasse c gehört,
- t_{ci}: tatsächlicher Wert (Ground Truth) für Pixel i in Klasse c,
- N: Anzahl der Pixel,
- ϵ: kleiner numerischer Wert zur numerischen Stabilisierung (z. B. $\epsilon = 1 \times 10^{-6}$).

Der Dice Loss für eine Klasse c berechnet sich folgendermaßen (4.2) [Planche and Andres, 2019]:

$$\text{DiceLoss}_c = 1 - Dice_c \tag{4.2}$$

Der gesamte Dice Loss für alle relevanten Klassen ergibt sich aus dem Mittelwert der Dice Loss-Werte aller Klassen (4.3):

$$\text{DiceLoss}_{\text{gesamt}} = \frac{1}{C} \sum_{c=1}^{C} \text{DiceLoss}_c \tag{4.3}$$

Dabei bezeichnet C die Anzahl der zu segmentierenden Klassen.

Cross-Entropy Loss

Die Cross-Entropy Loss misst, wie gut die vorhergesagten Klassenwahrscheinlichkeiten die tatsächlichen Klassen widerspiegeln, und ist definiert durch (4.4) [Planche and Andres, 2019]:

$$\text{CrossEntropyLoss} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} t_{ci} \cdot \log(p_{ci} + \epsilon) \tag{4.4}$$

Dabei gelten die oben genannten Definitionen der Symbole.

Kombinierte Verlustfunktion

Die hier verwendete kombinierte Verlustfunktion vereint beide beschriebenen Verlustfunktionen und kann durch Gewichtungsparameter angepasst werden. Formal wird sie wie folgt definiert (4.5) [Planche and Andres, 2019]:

$$\text{Gesamtverlust} = \alpha \cdot \text{CrossEntropyLoss} + \beta \cdot \text{DiceLoss}_{\text{gesamt}} \tag{4.5}$$

Dabei (4.5) sind α und β Hyperparameter, die bestimmen, wie stark die einzelnen Verluste zur Gesamtverlustfunktion beitragen. In dieser Arbeit wurden beide Verlustfunktionen gleich gewichtet ($\alpha = 1$, $\beta = 1$), um ein ausgewogenes Verhältnis zwischen pixelbasierter Genauigkeit (Cross-Entropy) und struktureller Übereinstimmung (Dice Loss) sicherzustellen.

Diese kombinierte Verlustfunktion ermöglicht insbesondere bei der Segmentierung von Zahnstrukturen eine höhere Präzision und robustere Segmentierungsergebnisse.

4.2.2.2. Berechnung der Klassengewichte

Um das Klassenungleichgewicht (*class imbalance*) in den annotierten Segmentierungsmasken auszugleichen, wurde in dieser Arbeit eine spezifische Gewichtungsstrategie für die Verlustfunktion verwendet. Diese basiert auf einer Berechnung der sogenannten Klassengewichte (*class weights*), welche aus der Häufigkeit der Pixel pro Klasse abgeleitet werden.

Die Berechnung der Klassengewichte erfolgte durch Zählung der Pixel je Klasse über alle Segmentierungsmasken im Datensatz. Dabei wurden ausschließlich Masken verwendet, die zur Segmentierung der Oberkieferzähne bestimmt sind (Dateien mit dem Zusatz „upper“). Pixel, die nicht zu einer der definierten Zielklassen gehören, wurden der Hintergrundklasse (Index 0) zugeordnet. Die Zielklassen entsprechen den Oberkieferzähnen der FDI-Klassifikation (11 bis 18 sowie 21 bis 28) und erhielten entsprechend fortlaufend die Indizes 1 bis 16.

Die Pixelanzahl für jede Klasse c wird formal definiert durch (4.6) [Planche and Andres, 2019]:

$$N_c = \sum_{i=1}^{M} \sum_{p \in I_i} \mathbf{1}(p = c) \tag{4.6}$$

Dabei sind:

- N_c: Gesamtzahl der Pixel, die Klasse c angehören,
- M: Gesamtanzahl der Masken im Datensatz,

- I_i: Menge aller Pixel in der Maske i,
- $\mathbf{1}(p = c)$: Indikatorfunktion, welche 1 zurückgibt, wenn der Pixelwert p der Klasse c entspricht, andernfalls 0.

Die Klassengewichte wurden anschließend als inverse Klassenhäufigkeit berechnet und normalisiert (4.7) [Planche and Andres, 2019]:

$$w_c = \frac{\frac{1}{N_c+\epsilon}}{\sum_{j=0}^{C} \frac{1}{N_j+\epsilon}} \tag{4.7}$$

Dabei gilt:

- w_c: Gewicht der Klasse c,
- C: Gesamtzahl der Klassen einschließlich Hintergrundklasse (hier 17: Hintergrund und 16 Zielklassen),
- ϵ: kleiner numerischer Wert zur Vermeidung einer Division durch null (z. B. $\epsilon = 1 \times 10^{-6}$).

Die ermittelten Klassengewichte wurden schließlich in einem PyTorch-Tensor gespeichert und beim Training der Modelle verwendet. Diese Gewichtungsstrategie ermöglicht eine bessere Ausgewogenheit bei der Modellierung und trägt wesentlich dazu bei, die Präzision und Robustheit der Segmentierung zu erhöhen.

4.2.2.3. Early Stopping

Early Stopping ist eine bewährte Methode, die in der Optimierung von Trainingsprozessen, insbesondere beim Deep Learning, angewendet wird [Planche and Andres, 2019]. Ziel dieser Technik ist es, den Trainingsprozess automatisch zu beenden, sobald keine merkliche Verbesserung der Modellleistung auf einem Validierungsdatensatz mehr festgestellt wird. Die Hauptaufgabe von Early Stopping besteht darin, eine Überanpassung (engl. *Overfitting*) zu verhindern. Überanpassung tritt auf, wenn das Modell die Trainingsdaten zu genau abbildet und dadurch die Fähigkeit verliert, auf neue, unbekannte Daten gut zu generalisieren [Planche and Andres, 2019].

Im Rahmen dieser Arbeit wurde Early Stopping mithilfe eines sogenannten Geduldsparameters (*patience*) implementiert. Dieser Parameter definiert, wie viele aufeinanderfolgende Epochen ohne Verbesserung der Validierungsleistung abgewartet werden, bevor der Trainingsprozess beendet wird.

Formal wird der Early-Stopping-Mechanismus wie folgt beschrieben:

Sei $V_{\text{dice}}^{(t)}$ der Wert der Evaluationsmetrik (z. B. Dice-Koeffizient) auf dem Validierungsdatensatz nach Epoche t, und sei $V_{\text{dice}}^{(\text{best})}$ der beste bisher erreichte Wert dieser Metrik. Dann gilt für einen internen Zähler (engl. *counter*):

$$\text{counter} = \begin{cases} \text{counter} + 1, & \text{falls } V_{\text{dice}}^{(t)} \leq V_{\text{dice}}^{(\text{best})} \\ 0, & \text{falls } V_{\text{dice}}^{(t)} > V_{\text{dice}}^{(\text{best})} \end{cases} \tag{4.8}$$

Sobald der Zähler die festgelegte Geduld (*patience*) überschreitet, d. h. wenn gilt:

$$\text{counter} \geq \text{patience}, \tag{4.9}$$

wird der Trainingsprozess gestoppt, und die Gewichte des Modells werden auf den Stand der Epoche mit der besten Validierungsleistung zurückgesetzt.

Die Verwendung von Early Stopping bietet mehrere Vorteile. Es reduziert signifikant die Trainingsdauer, spart Rechenressourcen und hilft, die Generalisierungsfähigkeit des trainierten Modells auf unbekannte Daten zu verbessern. Insbesondere bei komplexen und ressourcenintensiven Aufgaben wie der Zahnsegmentierung, welche im Fokus dieser Arbeit steht, erwies sich Early Stopping als äußerst effektive und zuverlässige Strategie.

4.2.2.4. Verwendeter Optimierer: Adam

In dieser Arbeit wurde als Optimierungsalgorithmus für das Training der verwendeten Deep-Learning-Modelle der Adam-Optimierer eingesetzt. Adam (*Adaptive Moment Estimation*) ist ein weit verbreiteter Optimierungsalgorithmus, der von Kingma und Ba im Jahr 2014 vorgeschlagen wurde. Er kombiniert die Vorteile zweier anderer Optimierungsverfahren, nämlich Momentum-basierter Verfahren und adaptiver Lernratenverfahren (wie AdaGrad und RMSProp) [Géron, 2022].

Adam berechnet adaptive Lernraten für jedes Modellparameter und berücksichtigt dabei sowohl den ersten Moment (den Mittelwert der Gradienten, ähnlich dem Momentum-Verfahren) als auch den zweiten Moment (die Varianz der Gradienten, ähnlich RMSProp). Dadurch gelingt es Adam, sowohl die Richtung als auch die Schrittgröße der Parameterupdates während des Trainings optimal anzupassen. Diese Eigenschaften führen dazu, dass Adam in der Praxis häufig schneller konvergiert und robuster gegenüber der Wahl der initialen Lernrate ist als herkömmliche Optimierer wie SGD (*Stochastic Gradient Descent*).

Formal lässt sich das Parameter-Update bei Adam für einen Parameter θ wie folgt beschreiben [Géron, 2022]:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t \tag{4.10}$$

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2 \tag{4.11}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \tag{4.12}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{4.13}$$

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \tag{4.14}$$

Hierbei stehen:

- m_t für den exponentiell gewichteten Durchschnitt der Gradienten (erster Moment).
- v_t für den exponentiell gewichteten Durchschnitt der quadratischen Gradienten (zweiter Moment).
- $\hat{m}_t$ und $\hat{v}_t$ für die jeweiligen bias-korrigierten Momente.
- g_t für den Gradienten zur Zeit t.
- β_1 und β_2 (typischerweise 0,9 und 0,999) für die exponentiellen Verfallsraten der Momente.
- η (Lernrate) und ϵ (typischerweise 10^{-8}) für Stabilität bei der Division.

Die Wahl des Adam-Optimierers in dieser Arbeit erfolgte aufgrund seiner Fähigkeit, schnell und zuverlässig zu konvergieren sowie seiner Robustheit gegenüber der Lernrate. Dies ermöglichte ein effizientes Training der komplexen Segmentierungsmodelle bei gleichzeitig guten Generalisierungseigenschaften.

4.2.2.5. Verwendung des GradScalers

Im Rahmen dieser Arbeit wurde der sogenannte GradScaler (*Gradient Scaler*) verwendet, welcher ein wichtiger Bestandteil des sogenannten *Mixed Precision*-Trainings ist. Mixed Precision beschreibt das Training neuronaler Netze mit reduzierter numerischer Präzision, typischerweise im 16-Bit-Floating-Point-Format (`float16`), anstelle des üblichen 32-Bit-Formats (`float32`) [Géron, 2022]. Dies reduziert den Speicherverbrauch erheblich und beschleunigt das Training, insbesondere auf GPUs der neueren Generationen.

Der GradScaler dient dabei dazu, die Gradienten während des Trainings künstlich hochzuskalieren, um einem numerischen Unterlauf (*underflow*) vorzubeugen, der typischerweise durch die geringere Genauigkeit von `float16` entstehen kann [Géron, 2022]. Insbesondere sehr kleine Gradientenwerte könnten sonst auf null abgerundet werden, wodurch wichtige Updates verloren gehen könnten und das Training stagniert oder instabil wird.

Die Funktionsweise des GradScalers lässt sich vereinfacht wie folgt beschreiben:

- Während des Vorwärtsdurchlaufs (*forward pass*) skaliert der GradScaler die Verluste künstlich hoch, wodurch auch die Gradienten entsprechend größer werden.
- Anschließend, im Rückwärtsdurchlauf (*backward pass*), werden diese skalierten Gradienten verwendet, um die Parameterupdates zu berechnen.
- Vor der eigentlichen Parameteraktualisierung (*optimizer step*) werden die Gradienten wieder auf die ursprüngliche Größenordnung zurückskaliert.

Die Verwendung des GradScalers ermöglicht somit ein stabiles, effizientes und schnelles Training neuronaler Netzwerke mit Mixed Precision und verbessert dabei maßgeblich die Leistung und Robustheit des Trainingsprozesses.

4.2.2.6. Fine-Tuning Strategie

In dieser Studie wurde zusätzlich zur Standard-Trainingsstrategie eine gezielte Fine-Tuning-Strategie implementiert und evaluiert. Beim Fine-Tuning wird ein bereits vortrainiertes neuronales Netzwerk zunächst teilweise „eingefroren“ (freeze) und anschließend schrittweise wieder geöffnet (unfreeze), um gezielt und kontrolliert Anpassungen an der spezifischen Aufgabenstellung vorzunehmen. Die implementierte Fine-Tuning Strategie umfasste folgende Schritte:

1. **Initiale Freezing-Phase:** Zu Beginn wurden alle Parameter des Encoder-Moduls eingefroren, indem ihre `requires_grad`-Eigenschaft auf `False` gesetzt wurde. Dies bewirkte, dass anfangs nur der Decoder trainiert wurde und der Encoder seine vortrainierten Merkmale zunächst behielt.
2. **Schrittweise Unfreezing-Phase:** Im Verlauf des Trainings wurden schrittweise einzelne Encoder-Schichten zu festgelegten Epochen (z.B. Epoch 15, 30, 45) geöffnet, um gezielt Anpassungen vorzunehmen und dabei Überanpassungseffekte zu minimieren.

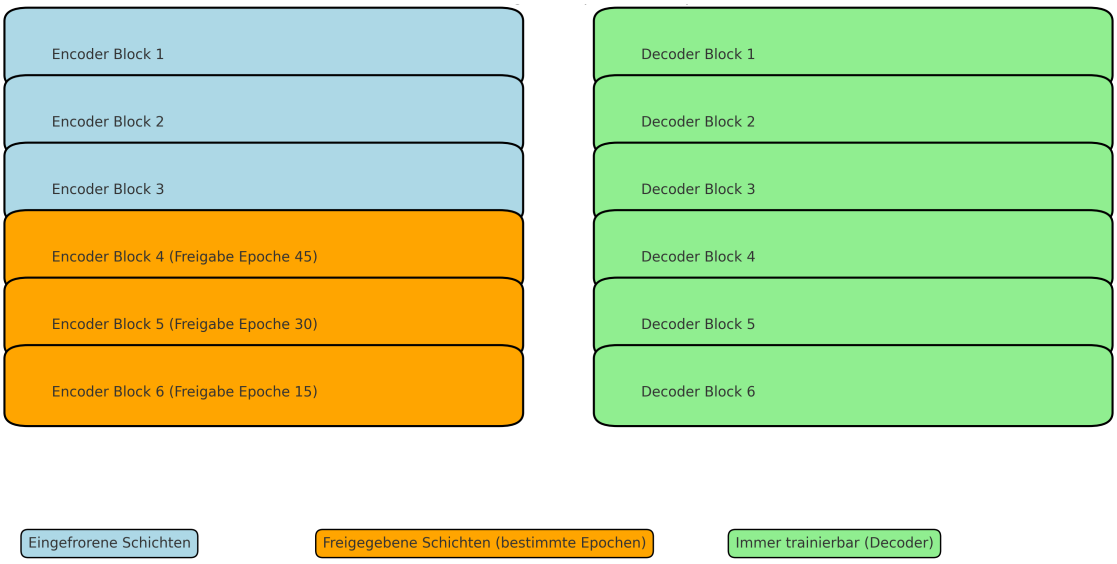


Abbildung 4.7.: Schematische Darstellung der verwendeten Fine-Tuning-Strategie.

Trotz der theoretischen Vorteile dieser kontrollierten Anpassungsstrategie zeigte sich in der Praxis, dass die Fine-Tuning-Variante gegenüber der vollständig trainierten Standardvariante (bei der von Beginn an alle Modellparameter trainiert wurden) schwächere Segmentierungsergebnisse lieferte.

Die vollständig trainierte UNet-Variante (ohne Fine-Tuning) erzielte in den Experimenten konsistent höhere Werte insbesondere beim Dice-Koeffizienten und der Gesamtgenauigkeit. Daher wurde abschließend die Standard-UNet-Trainingsstrategie bevorzugt und für die finale Modellwahl eingesetzt.

4.2.2.7. Weitere Trainingsdetails

Neben den bereits erläuterten Methoden und Optimierungstechniken wurden zusätzliche spezifische Details im Trainingsprozess berücksichtigt:

- **Lernrate (Learning Rate)**: Die Lernrate wurde initial auf \$0,0001\$ festgelegt. Diese niedrige Lernrate ermöglicht eine präzise und stabile Anpassung der Gewichte während des Trainings.
- **Batchgröße (Batch Size)**: Als Batchgröße wurde 8 verwendet, um einen optimalen Kompromiss zwischen Trainingsgeschwindigkeit und Speicherverbrauch zu erzielen.
- **Datenaugmentation**: Während des Trainings wurde umfangreiche Datenaugmentation durchgeführt, einschließlich zufälliger Drehungen, Skalierungen, hori-

zontaler und vertikaler Spiegelungen, um die Generalisierungsfähigkeit der Modelle zu verbessern.

Diese zusätzlichen Details tragen dazu bei, das Training effizient und robust durchzuführen und somit optimale Segmentierungsergebnisse sicherzustellen.

4.3. Experimente

In diesem Abschnitt wird das experimentelle Vorgehen ausführlich beschrieben. Dies umfasst die Definition der verwendeten Verlustfunktion, die Berechnung der Klassengewichte zur Optimierung des Trainingsprozesses sowie die Implementierung des Early-Stopping-Verfahrens zur Vermeidung von Überanpassung. Zusätzlich werden hier weitere Details zur Trainingsdurchführung und zu den angewendeten Optimierungsstrategien erläutert.

4.3.1. Durchführung der Experimente

Die Experimente zur Evaluierung der verwendeten Modelle wurden nach einem systematischen und einheitlichen Ablauf durchgeführt, um sowohl die Vergleichbarkeit der Ergebnisse sicherzustellen als auch eine verlässliche und reproduzierbare Grundlage für die Auswertung zu schaffen.

Zunächst wurden alle zuvor beschriebenen Modelle (UNet, BiSeNetV2, DenseASPP, SegNet, BASNet und Mask R-CNN) unter gleichen experimentellen Bedingungen auf dem erstellten 2D-Datensatz trainiert. Für jedes Modell wurden identische Datensatzaufteilungen verwendet: 80 % der Daten für das Training, 10 % für die Validierung und 10 % für den Test.

Während des Trainingsprozesses wurden folgende Abläufe beachtet:

- Nach jeder Epoche wurde die Leistung der Modelle auf dem Validierungsdatensatz anhand des Dice-Koeffizienten bewertet.
- Die Modelle wurden jeweils so lange trainiert, bis entweder die maximale Anzahl an Epochen (standardmäßig 50 Epochen) erreicht oder der Trainingsprozess durch Early Stopping vorzeitig beendet wurde. Early Stopping wurde aktiviert, wenn über 10 aufeinanderfolgende Epochen keine Verbesserung der Validierungsleistung beobachtet werden konnte.
- Zur Sicherstellung der Reproduzierbarkeit und zur bestmöglichen Leistungsbewertung wurde das Modell mit der besten Leistung auf dem Validierungsdatensatz gespeichert (*Checkpointing*) und anschließend für die finale Evaluierung auf dem Testdatensatz verwendet.

Für die Auswertung der Ergebnisse wurden sowohl quantitative als auch qualitative Analysen durchgeführt:

- **Quantitative Analyse**: Die finale Leistungsbewertung erfolgte über die Berechnung des Dice-Koeffizienten auf dem unabhängigen Testdatensatz. Diese Metrik

ermöglicht eine objektive Bewertung der Genauigkeit und Robustheit der Segmentierung.

- **Qualitative Analyse**: Zusätzlich wurden exemplarische Segmentierungsergebnisse visuell überprüft und dargestellt. Dabei wurde besonders auf die präzise Abbildung anatomischer Strukturen sowie die korrekte und scharfe Trennung der Zähne geachtet.

Durch diesen strukturierten und detaillierten Versuchsablauf wurde gewährleistet, dass die Ergebnisse zuverlässig sind und eindeutige Aussagen über die Eignung und Leistungsfähigkeit der untersuchten Modelle bei der Zahnsegmentierung getroffen werden konnten.

4.3.2. Evaluationsmetriken

Zur umfassenden Bewertung der Segmentierungsergebnisse wurden neben dem bereits beschriebenen **Dice-Koeffizienten** (2.22) weitere wichtige Evaluationsmetriken eingesetzt, welche die Leistung der Modelle detaillierter beschreiben.

Vordergrund-Genauigkeit (Foreground-Accuracy)

Die Vordergrund-Genauigkeit bewertet die Genauigkeit der Modellvorhersagen ausschließlich auf relevanten Bereichen (den Zähnen):

$$\text{Vordergrund-Accuracy} = \frac{\text{Anzahl korrekt klassifizierter Vordergrund-Pixel}}{\text{Gesamtzahl der Vordergrund-Pixel}} \tag{4.15}$$

Diese Metrik ist entscheidend, da sie spezifisch die Genauigkeit der segmentierten anatomischen Strukturen bewertet und die Hintergrundbereiche ausklammert.

Hintergrund-Genauigkeit (Background-Accuracy)

Die Hintergrund-Genauigkeit misst, wie zuverlässig das Modell nicht relevante Bereiche (Hintergrund) erkennt und korrekt klassifiziert:

$$\text{Hintergrund-Accuracy} = \frac{\text{Anzahl korrekt klassifizierter Hintergrund-Pixel}}{\text{Gesamtzahl der Hintergrund-Pixel}} \tag{4.16}$$

Eine hohe Hintergrund-Genauigkeit weist darauf hin, dass das Modell präzise zwischen Hintergrund und relevanten Strukturen differenzieren kann.

Background False Positive Rate (Hintergrund-Falsch-Positiv-Rate)

Diese Metrik quantifiziert, welcher Anteil des Hintergrundes fälschlicherweise als Vordergrund (Zähne) erkannt wird:

$$\text{Background False Positive Rate} = \frac{\text{Anzahl falsch als Vordergrund klassifizierter Hintergrund-Pixel}}{\text{Gesamtzahl der Hintergrund-Pixel}} \tag{4.17}$$

Ein niedriger Wert zeigt an, dass das Modell nur wenige Fehlklassifikationen im Hintergrundbereich verursacht.

Precision, Recall und F1-Score (Vordergrund)

Zur Bewertung der binären Klassifikationsleistung des Modells zwischen Vordergrund (Zähne) und Hintergrund wurden folgende gängige Metriken verwendet:

Precision (Genauigkeit) (2.23) [Géron, 2022]:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{4.18}$$

Recall (Sensitivität) (2.24) [Géron, 2022]:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{4.19}$$

F1-Score (harmonisches Mittel von Precision und Recall) (2.25) [Géron, 2022]:

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.20}$$

Hierbei gelten folgende Definitionen:

- **TP (True Positive)**: Vordergrundpixel, die vom Modell korrekt als Vordergrund erkannt wurden.
- **FP (False Positive)**: Hintergrundpixel, die fälschlicherweise als Vordergrund erkannt wurden.
- **FN (False Negative)**: Vordergrundpixel, die nicht erkannt und somit fälschlicherweise als Hintergrund klassifiziert wurden.

Insbesondere der F1-Score bietet eine ausgewogene Bewertung, da er sowohl falsch-positive als auch falsch-negative Klassifikationen gleichermaßen berücksichtigt.

4.3.3. Ergebnisse der Experimente

In diesem Abschnitt werden die Ergebnisse der durchgeführten Experimente quantitativ und qualitativ dargestellt.

4.3.3.1. Quantitative Ergebnisse

Die Ergebnisse der quantitativen Evaluation der verwendeten Modelle sind in Tabelle 4.3 zusammengefasst. Die Modelle wurden anhand folgender Evaluationsmetriken bewertet: Dice-Koeffizient, Foreground-Accuracy (Fg_Acc), Background False Positive Rate (Bg_FP_Rate), Precision (Fg), Recall (Fg) und F1-Score (Fg).

Tabelle 4.3.: Quantitative Ergebnisse der getesteten Modelle

Modell	**Dice**	**Fg_Acc**	**Bg_FP_Rate**	**Precision (Fg)**	**Recall (Fg)**	**F1-Score (Fg)**
UNet	0,7400	0,8742	0,0082	0,7651	0,9219	0,8200
SegNet	0,7031	0,8872	0,0097	0,7379	0,9379	0,8101
BASNet	**0,7610**	**0,8882**	**0,0076**	**0,7740**	0,9272	**0,8270**
BiSeNet	0,5414	0,7181	0,0150	0,6553	0,9268	0,7507
DenseASPP	0,6135	0,8697	0,0131	0,6784	**0,9454**	0,7740

Aus den quantitativen Ergebnissen geht hervor, dass BASNet insgesamt die beste Performance aufweist, insbesondere im Hinblick auf den Dice-Koeffizienten und den F1-Score. UNet und SegNet zeigten ebenfalls gute Ergebnisse, während BiSeNet und DenseASPP relativ schwächere Leistungen erzielten.

4.3.3.2. Qualitative Ergebnisse (Visualisierung und Interpretation)

Die qualitativen Ergebnisse dienen der visuellen Beurteilung und Interpretation der Segmentierungsleistung. Insbesondere wurden hierbei die Präzision der anatomischen Grenzen sowie die Klarheit der Segmentierungen visuell beurteilt.

Folgende Abbildungen zeigen exemplarische Segmentierungsergebnisse der getesteten Modelle:

- **UNet**:

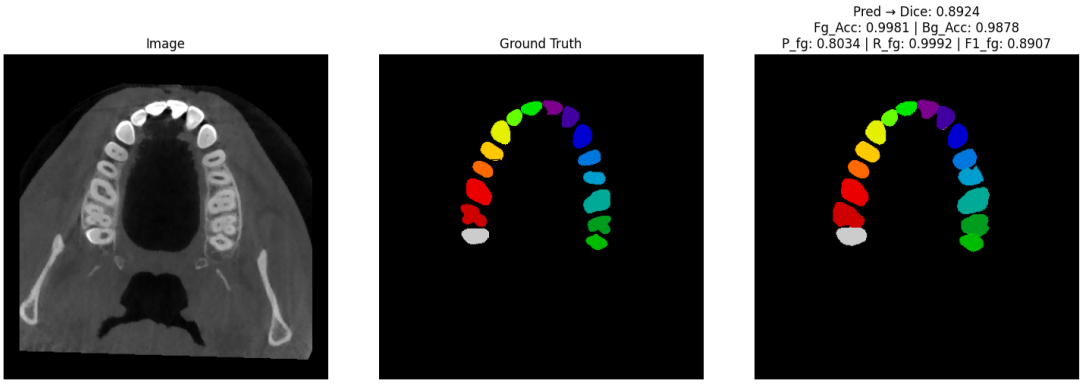


Abbildung 4.8.: Beispielhafte Segmentierungsergebnisse des UNet-Modells

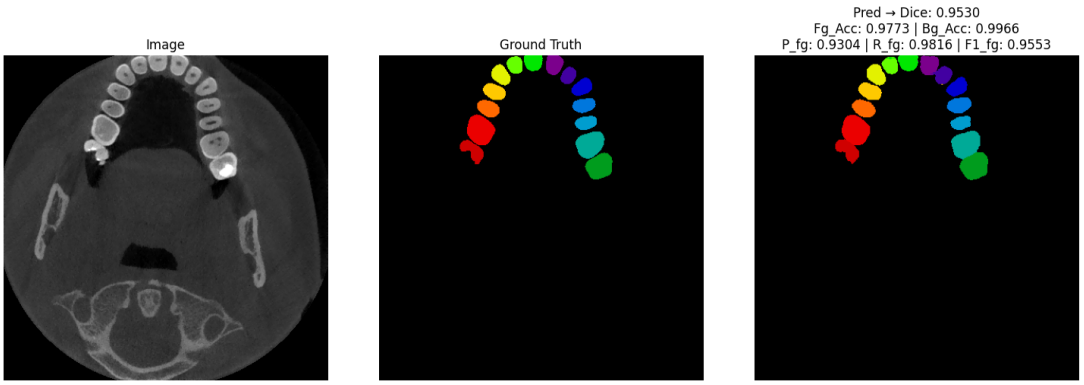


Abbildung 4.9.: Beispielhafte Segmentierungsergebnisse des UNet-Modells

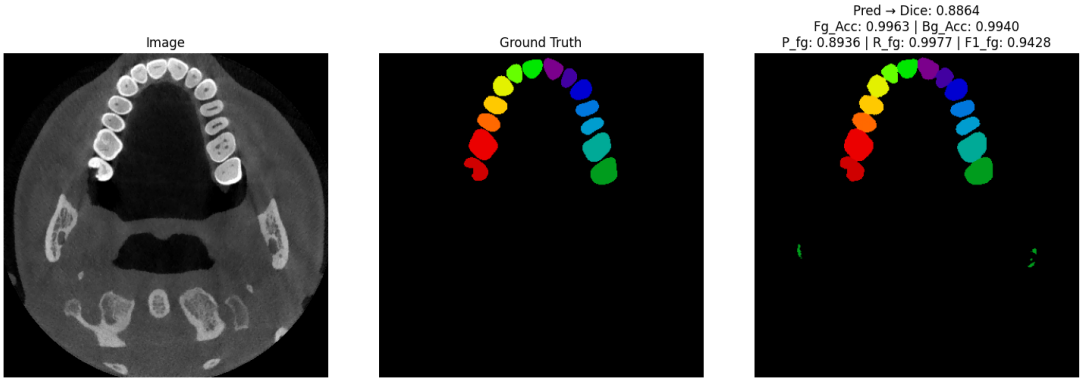


Abbildung 4.10.: Beispielhafte Segmentierungsergebnisse des UNet-Modells

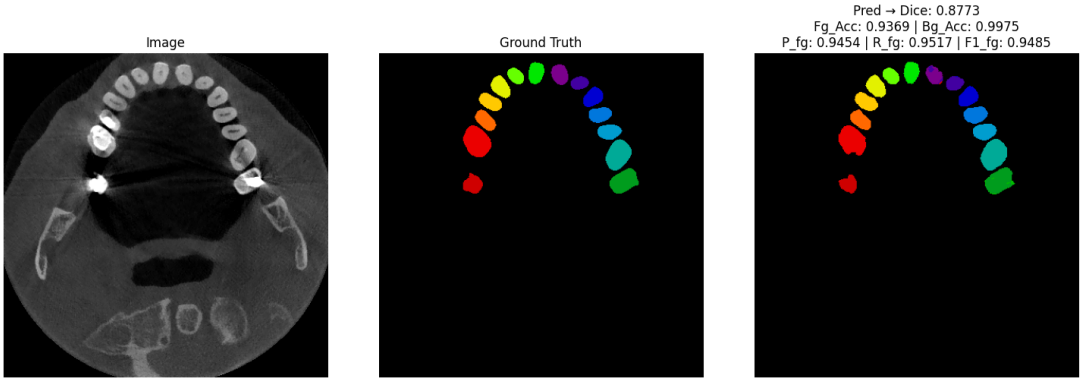


Abbildung 4.11.: Beispielhafte Segmentierungsergebnisse des UNet-Modells

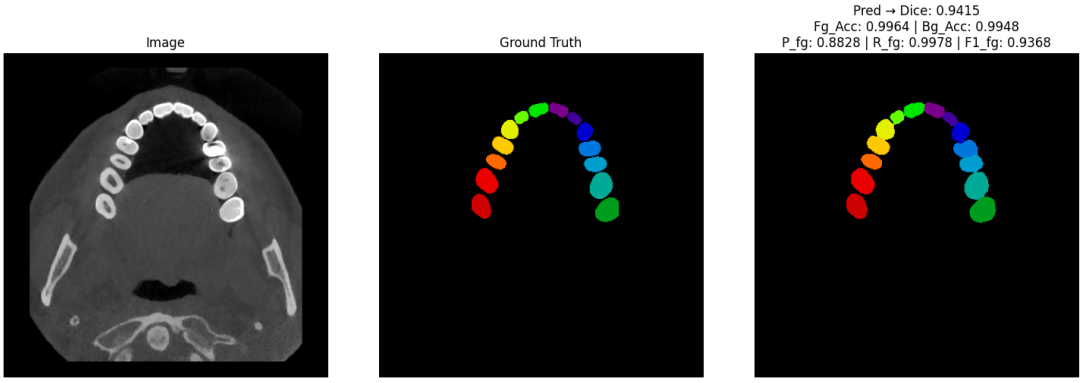


Abbildung 4.12.: Beispielhafte Segmentierungsergebnisse des UNet-Modells

- **SegNet**:

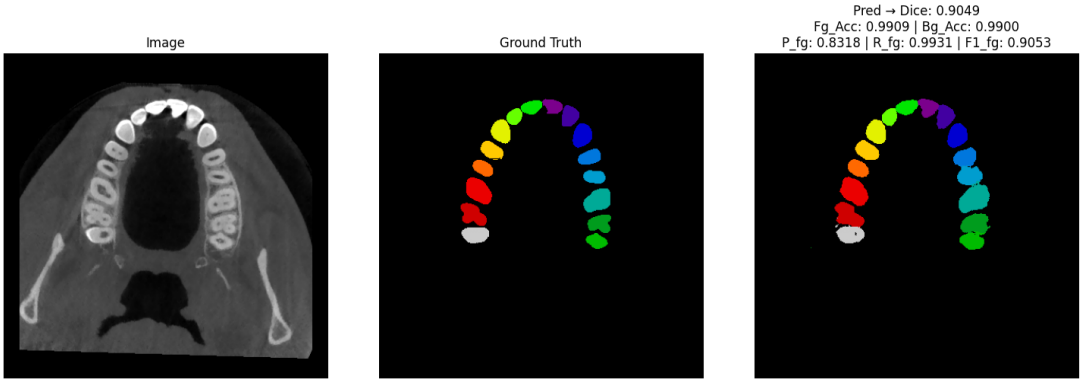


Abbildung 4.13.: Beispielhafte Segmentierungsergebnisse des SegNet-Modells

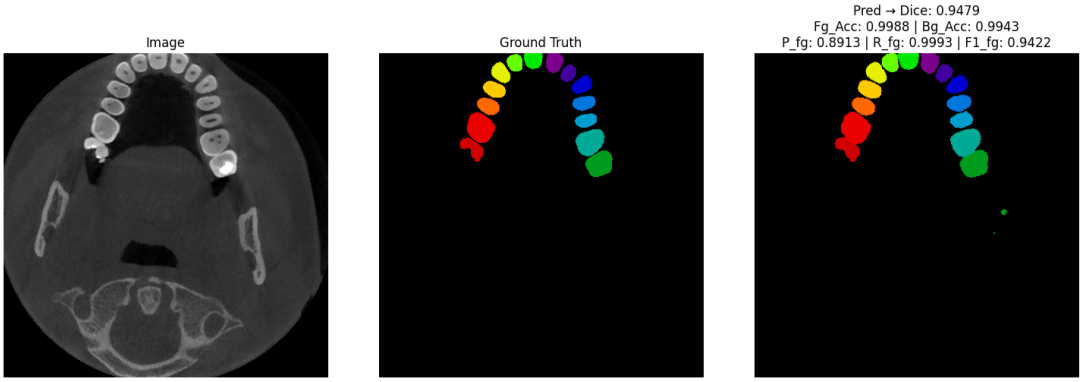


Abbildung 4.14.: Beispielhafte Segmentierungsergebnisse des SegNet-Modells

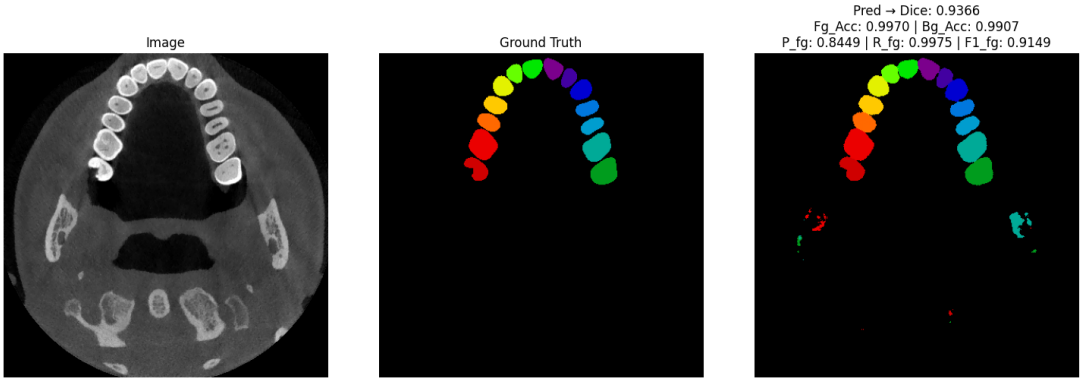


Abbildung 4.15.: Beispielhafte Segmentierungsergebnisse des SegNet-Modells

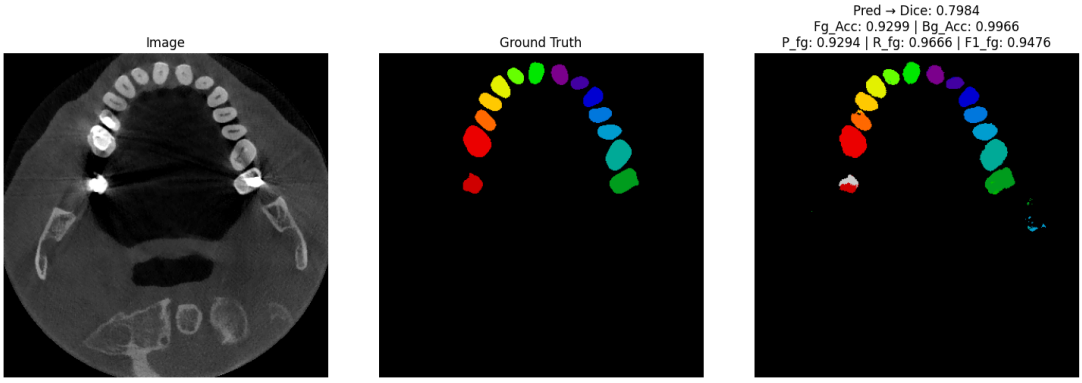


Abbildung 4.16.: Beispielhafte Segmentierungsergebnisse des SegNet-Modells

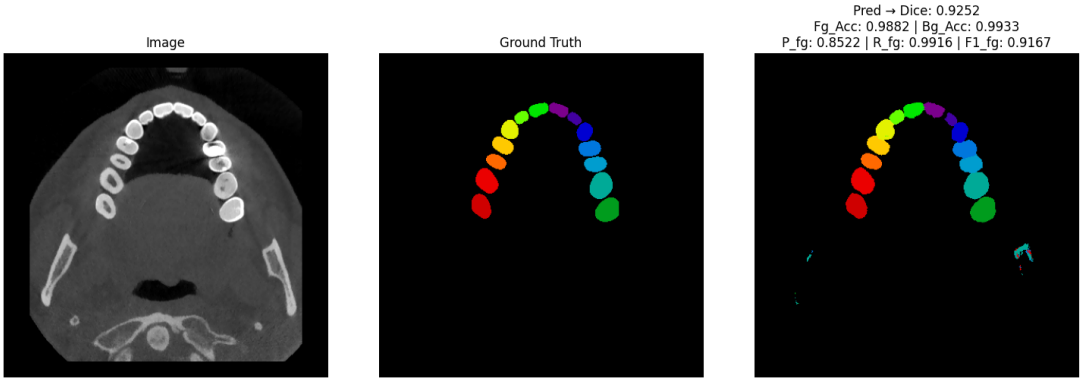


Abbildung 4.17.: Beispielhafte Segmentierungsergebnisse des SegNet-Modells

- **BASNet**:

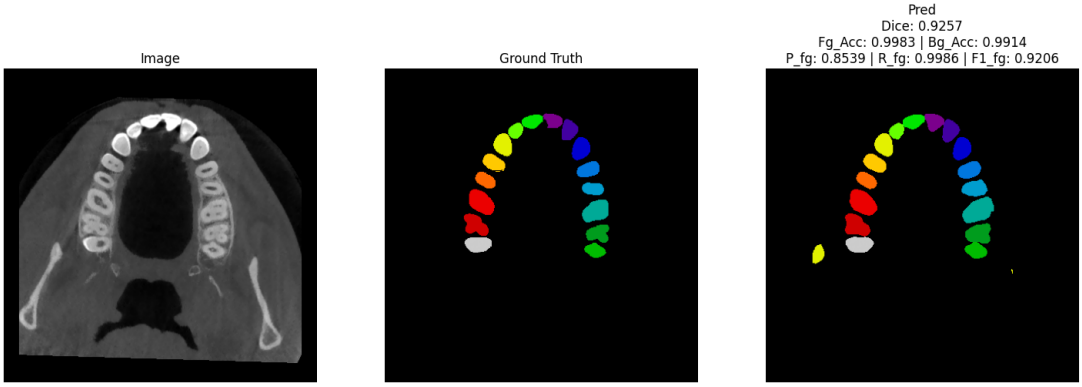


Abbildung 4.18.: Beispielhafte Segmentierungsergebnisse des BASNet-Modells

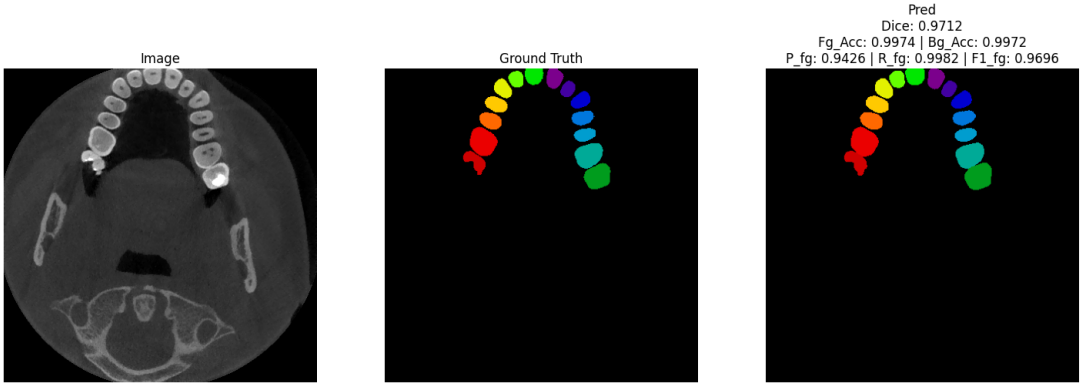


Abbildung 4.19.: eispielhafte Segmentierungsergebnisse des BASNet-Modells

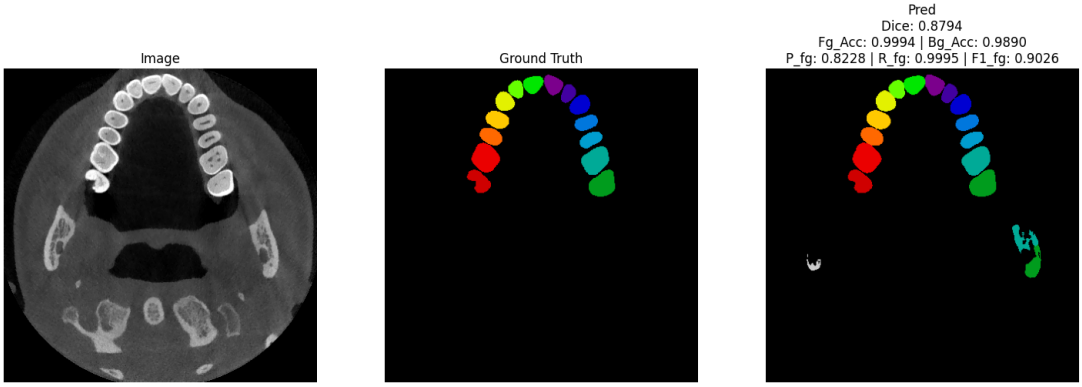


Abbildung 4.20.: Beispielhafte Segmentierungsergebnisse des BASNet-Modells

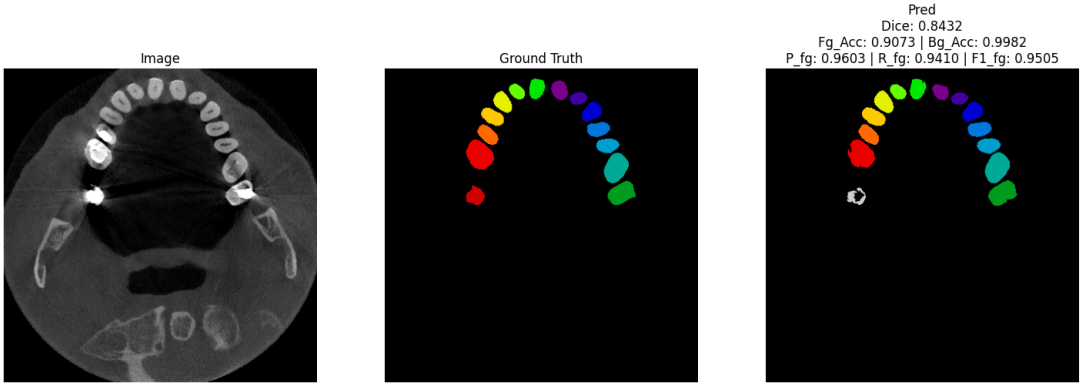


Abbildung 4.21.: Beispielhafte Segmentierungsergebnisse des BASNet-Modells

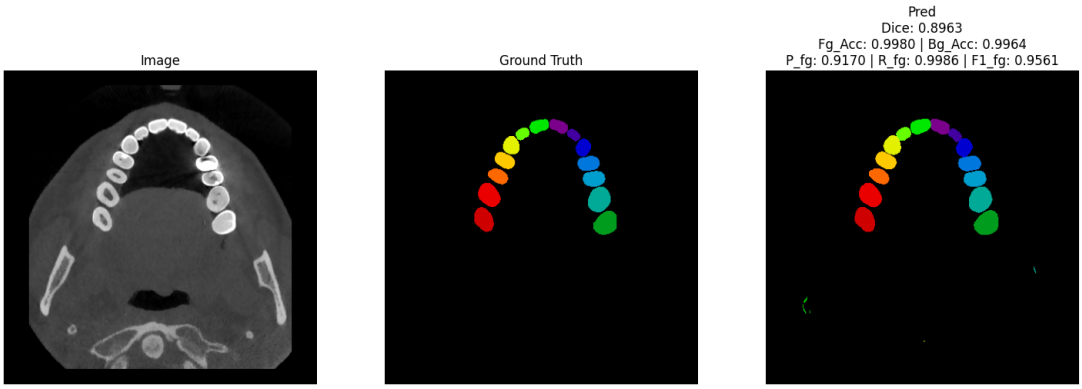


Abbildung 4.22.: Beispielhafte Segmentierungsergebnisse des BASNet-Modells

- **BiSeNet**:

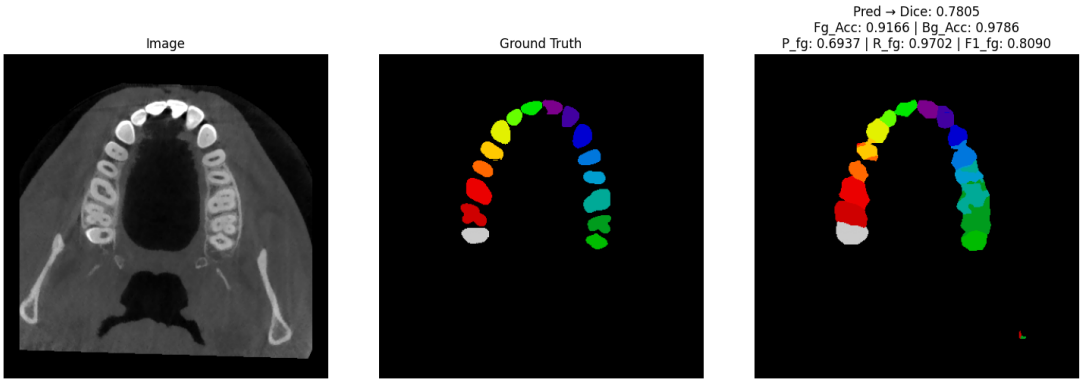


Abbildung 4.23.: Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells

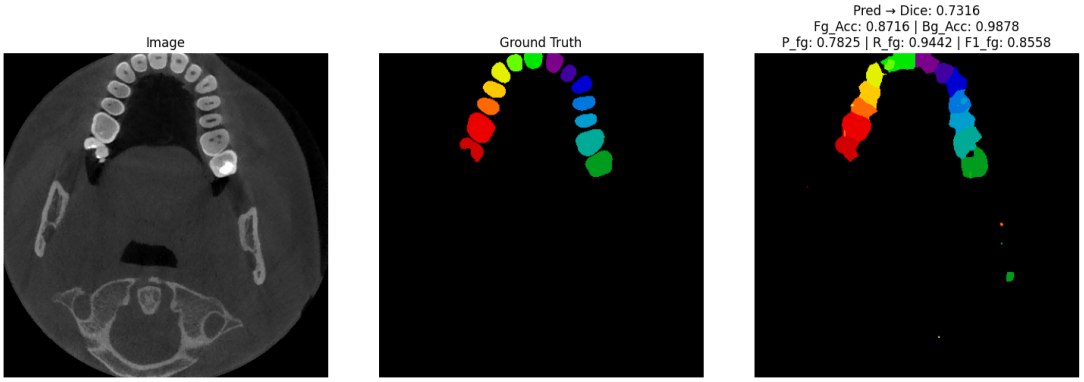


Abbildung 4.24.: Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells

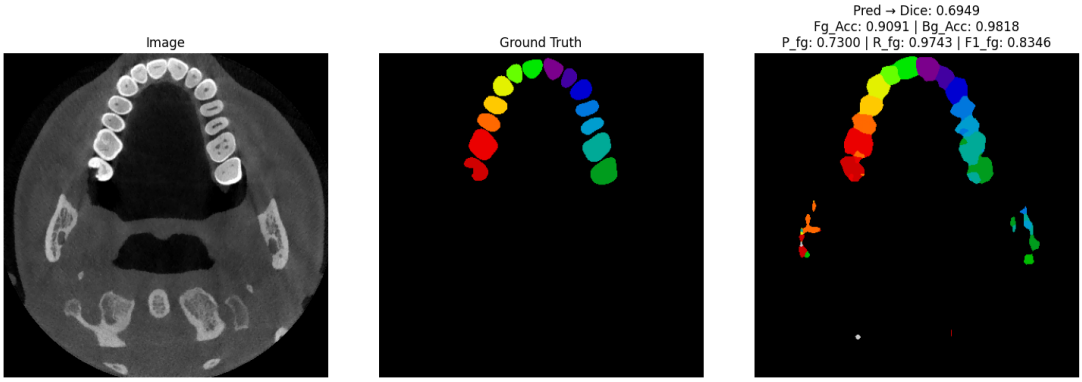


Abbildung 4.25.: Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells

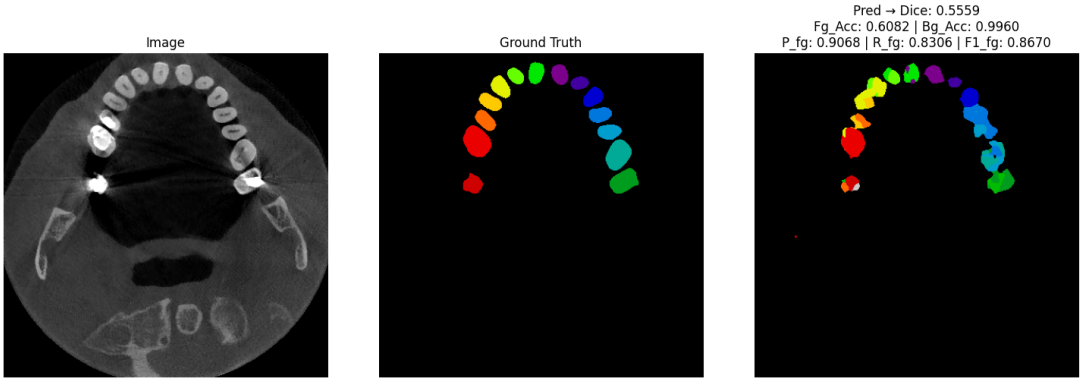


Abbildung 4.26.: Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells

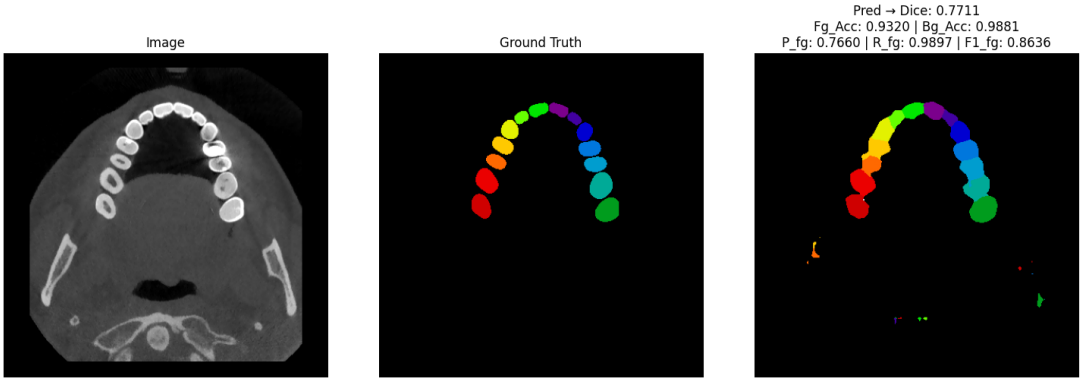


Abbildung 4.27.: Beispielhafte Segmentierungsergebnisse des BiSeNet-Modells

- **DenseASPP**:

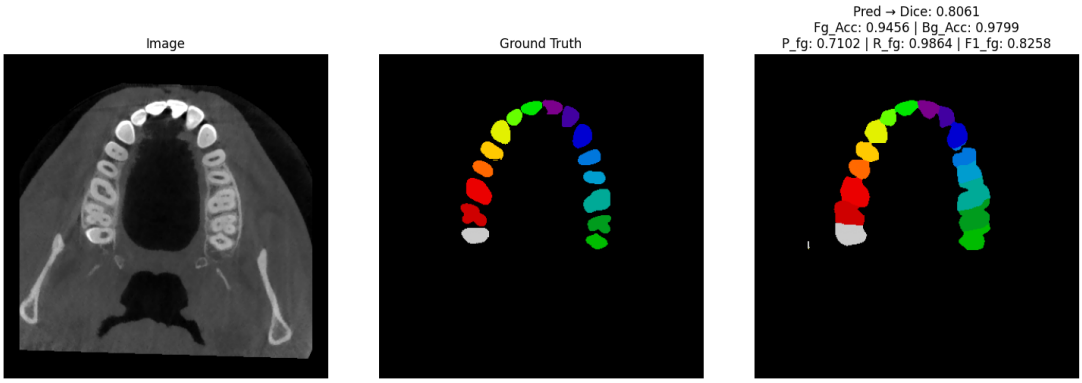


Abbildung 4.28.: Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells

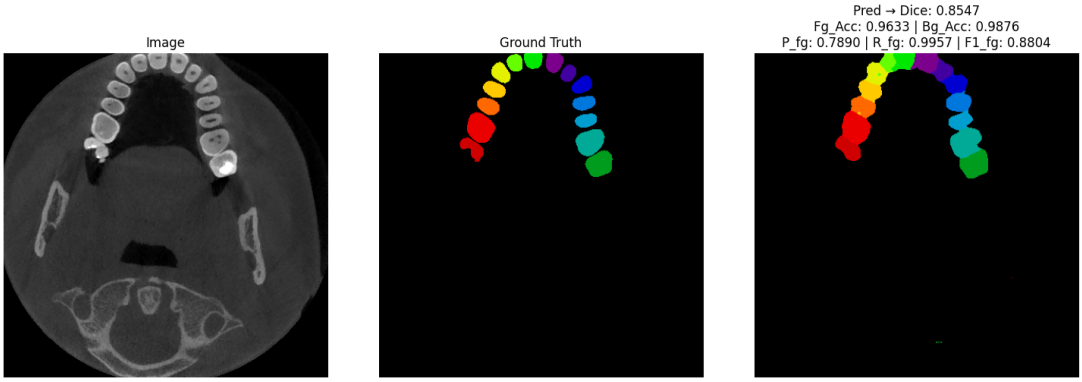


Abbildung 4.29.: Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells

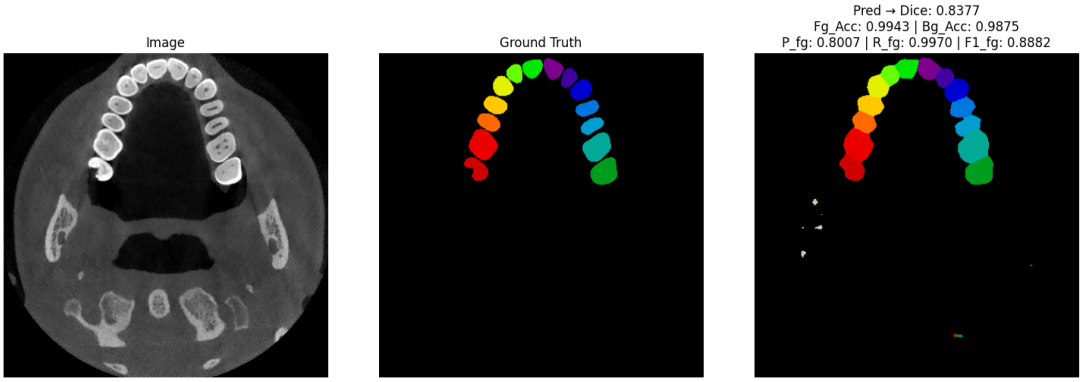


Abbildung 4.30.: Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells

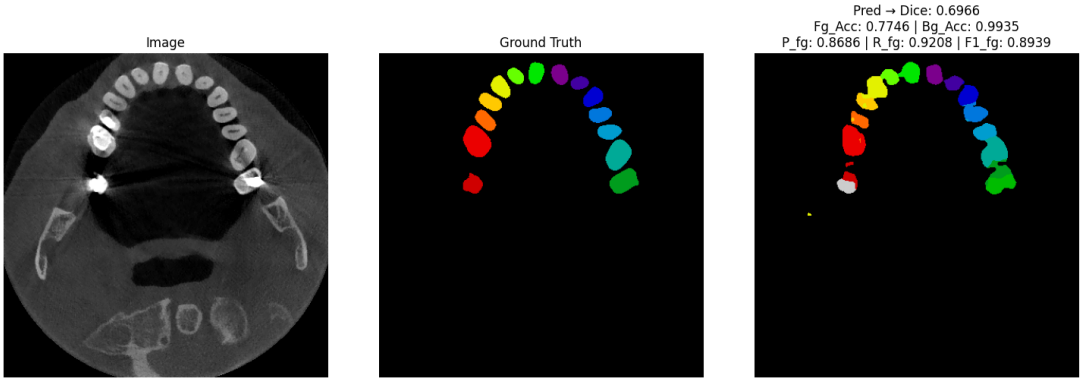


Abbildung 4.31.: Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells

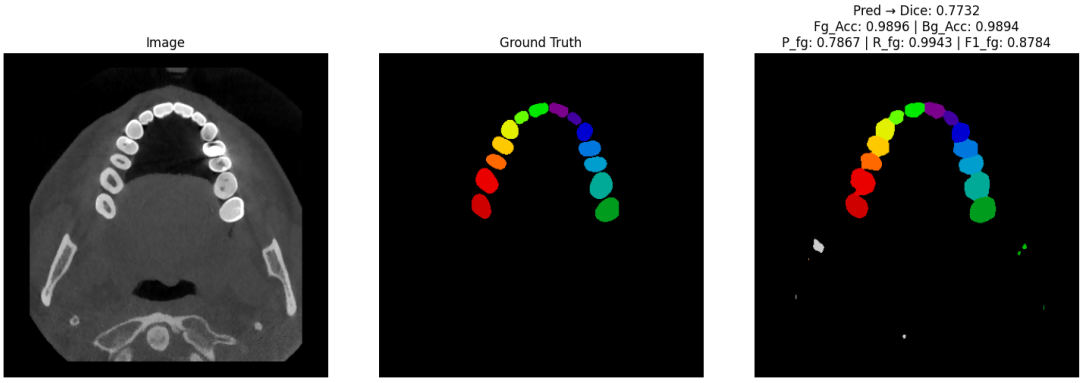


Abbildung 4.32.: Beispielhafte Segmentierungsergebnisse des DenseASPP-Modells

Ausführliche Analyse zum Scheitern des Mask R-CNN-Modells

Das Mask R-CNN-Modell lieferte unerwartet schlechte Ergebnisse, die deutlich unter den Erwartungen lagen. Erste Evaluationen zeigten, dass die Genauigkeit (Accuracy) des Modells bei der Segmentierung der Zähne in vielen Fällen unter 40 % lag, was auf erhebliche Probleme bei der korrekten Zuordnung und Abgrenzung der anatomischen Strukturen hindeutet. Aufgrund der Popularität und grundsätzlich hohen Leistungsfähigkeit des Mask R-CNN-Ansatzes wurden detaillierte Untersuchungen durchgeführt, um die Ursache dieser schlechten Ergebnisse zu ermitteln.

Zunächst wurde überprüft, ob ein Implementierungsfehler vorlag, insbesondere im Hinblick auf das Laden und die korrekte Zuordnung der Ground-Truth-Masken. Dabei wurde beispielsweise gezielt kontrolliert, ob alle Masken richtig geladen, segmentierte Klassen korrekt zugewiesen und entsprechende Annotationen pixelgenau übertragen wurden. Diese Untersuchungen bestätigten, dass die Masken ordnungsgemäß und ohne Fehler geladen wurden und somit ein Implementierungsfehler hinsichtlich der Datenzuführung ausgeschlossen werden konnte.

In einem weiteren Schritt wurde ein sogenannter Overfit-Test durchgeführt, um zu prüfen, ob das Modell grundsätzlich in der Lage ist, gegebene Trainingsdaten vollständig und korrekt zu lernen. Bei diesem Test wird das Modell auf einem bewusst sehr kleinen Datensatz (wenige Trainingsbeispiele, meist nur 1-3 Bilder) so lange trainiert, bis es diese Beispiele nahezu perfekt segmentieren kann. Ziel dieses Tests ist es, sicherzustellen, dass das Modell strukturell und algorithmisch überhaupt in der Lage ist, die Zielstrukturen zu lernen. Unerwarteterweise zeigte sich, dass das Mask R-CNN-Modell selbst diesen grundlegenden Test nicht bestand. Trotz längerer Trainingszeit und variierenden Hyperparametern (wie Lernrate, Batch-Größe, Regularisierungen usw.) konnte das Modell die wenigen Trainingsbeispiele nicht zufriedenstellend lernen und somit keine vollständige Anpassung (Overfitting) erzielen.

Das Scheitern des Overfit-Tests deutet darauf hin, dass das verwendete Mask R-CNN-Modell in der vorliegenden Konfiguration oder in Kombination mit den genutzten Datenstrukturen möglicherweise ungeeignet ist. Weitere mögliche Gründe könnten in der Struktur und Natur der verwendeten 2D-Slice-basierten Daten liegen, die nicht optimal zu den Anforderungen und Erwartungen von Mask R-CNN passen könnten. Mask R-CNN ist ursprünglich für Instanzsegmentierungen entwickelt worden, bei denen oft klar abgegrenzte, relativ unabhängige Objekte identifiziert werden müssen. Die verwendeten zahnmedizinischen Datensätze dagegen bestehen aus vielen kleinen, teils sehr eng nebeneinanderliegenden anatomischen Strukturen, was die korrekte Abgrenzung und Zuordnung erschwert und dem Mask R-CNN-Ansatz möglicherweise nicht gerecht wird.

Zusammenfassend lässt sich festhalten, dass trotz eingehender Untersuchungen kein offensichtlicher Implementierungsfehler gefunden wurde, das Modell jedoch grundsätzlich nicht in der Lage war, die vorhandenen Daten zufriedenstellend zu erlernen. Zukünftige Untersuchungen könnten alternative Trainingsstrategien, speziell angepasste Modellvarianten oder tiefere Analysen der Kompatibilität des Mask R-CNN-Modells mit ähnlichen medizinischen Datensätzen in Betracht ziehen.

4.3.4. Diskussion der Ergebnisse

In diesem Abschnitt werden die erzielten Ergebnisse interpretiert und diskutiert, die verwendeten Modelle miteinander verglichen sowie mögliche Limitationen und Verbesserungspotenziale der Studie erläutert.

4.3.4.1. Visuelle Bewertung der Segmentierungsergebnisse der Modelle

In diesem Abschnitt wurden die Segmentierungsergebnisse der Modelle auf visueller Basis bewertet. Die Bewertung erfolgte unabhängig von quantitativen Metriken und konzentrierte sich stattdessen auf anatomische Genauigkeit, Zahndifferenzierung, Konturschärfe und Hintergrundrauschen.

Das U-Net-Modell lieferte eine insgesamt qualitativ hochwertige und konsistente Segmentierung der Zähne. Die Zähne wurden sowohl hinsichtlich ihrer Position als auch ihrer Form korrekt identifiziert, und die Trennung zwischen den einzelnen Segmenten war deutlich. Die Übergänge waren scharf und weitgehend rauschfrei. In den hinteren Zahnregionen (Molarenbereich) wurden jedoch geringfügige Randunschärfen beobachtet.

Das SegNet-Modell zeigte ebenfalls zufriedenstellende Ergebnisse, allerdings mit etwas unregelmäßigeren Segmentierungsgrenzen im Vergleich zu U-Net. Die Zahnpositionen wurden korrekt erkannt, jedoch trat ein kleiner, fälschlicherweise segmentierter Bereich außerhalb des eigentlichen Zahnareals auf. Abgesehen von diesem Fehler war die Segmentierung insgesamt überzeugend.

Das DenseASPP-Modell konnte zwar die allgemeine Zahnform erfassen, wies jedoch mehrere Mängel auf. Einzelne Zahnsegmente fehlten vollständig oder waren nur teilweise vorhanden, insbesondere waren die Grenzen zwischen den Zähnen undeutlich. Darüber hinaus traten kleine, nicht zusammenhängende Bereiche außerhalb der Zahnstruktur auf, was auf eine gewisse Unsicherheit des Modells hinweist.

Die mit dem BiSeNet-Modell erzielten Ergebnisse zeigten die geringste visuelle Segmentierungsqualität. Zwar wurde die Zahnreihe grob erkannt, jedoch waren die Segmentgrenzen unscharf, und eine klare Trennung zwischen den Zähnen war häufig nicht

gegeben. Zudem war ein auffälliges Maß an falsch positiven Segmentierungen im Hintergrund sichtbar, was auf eine geringe Modellstabilität schließen lässt.

Das BASNet-Modell hingegen lieferte die visuell überzeugendsten Segmentierungsergebnisse. Die Zähne wurden vollständig, exakt und mit klar definierten Rändern segmentiert. Es gab keine Überschneidungen oder Unschärfen zwischen den einzelnen Zahnsegmenten. Zudem traten keinerlei falsch positive Segmentierungen auf, was auf eine hohe Trennschärfe gegenüber dem Hintergrund hinweist. Das Ergebnis des BASNet-Modells kann als klinisch und wissenschaftlich nutzbar eingestuft werden.

Zusammenfassend lässt sich feststellen, dass BASNet und U-Net im Vergleich zu den anderen Modellen die qualitativ besten visuell bewerteten Segmentierungsergebnisse lieferten. Beide Modelle zeigten eine hohe Genauigkeit bei der Lokalisierung und Trennung der Zähne sowie bei der Vermeidung von Hintergrundrauschen. Die übrigen Modelle wiesen in unterschiedlichen Ausprägungen Schwächen auf. Visuelle Genauigkeit und Konsistenz sind insbesondere in sensiblen Anwendungsbereichen wie der medizinischen Bildverarbeitung entscheidende Kriterien bei der Modellauswahl.

Bewertung der Modellperformance anhand der Trainings- und Validierungskurven:

1. UNet-Modell

Dice-Score:
Das UNet-Modell zeigt eine konsistente Lernkurve, bei der sowohl Trainings- als auch Validierungswerte stetig und stabil steigen. Der Trainings-Dice-Score erreicht etwa den Wert von 0,85, während der Validierungswert knapp darunter bei ca. 0,80 liegt (siehe Abbildung B.1 und B.2).
Die geringe Diskrepanz deutet auf eine gute Generalisierungsfähigkeit und geringe Überanpassung hin.
Loss-Werte:
Die Verlustfunktion zeigt eine schnelle Reduktion in Training und Validierung (siehe Abbildung B.1 und A.2). Der Validierungs-Loss stabilisiert sich nach etwa 30 Epochen.

UNet-Modell mit Fine-Tuning

Dice-Score:
Das feinjustierte UNet-Modell zeigt ebenfalls eine konsistente Lernkurve. Der Trainings-Dice erreicht etwa den Wert von 0,85, während der Validierungs-Dice knapp darunter stabil bei ca. 0,80 verbleibt (siehe Abbildung B.2).
Die Übereinstimmung der Kurven verdeutlicht eine geringe Tendenz zu Overfitting und eine gute Generalisierungsfähigkeit des feinjustierten Modells.

Loss-Werte:
Die Trainings- und Validierungsverluste sinken anfangs zügig und stabilisieren sich im späteren Trainingsverlauf (siehe Abbildung A.2). Der Validierungs-Loss bleibt auf moderatem Niveau und zeigt keine Anzeichen starker Überanpassung.

2. SegNet-Modell

Dice-Score:
SegNet erreicht rasch einen Trainings-Dice-Wert über 0,85, während der Validierungswert bei etwa 0,80 stagniert (siehe Abbildung B.3). Die erkennbare Diskrepanz weist auf ein gewisses Maß an Overfitting hin.
Loss-Werte:
Der Trainings-Loss fällt kontinuierlich, der Validierungs-Loss stagniert und steigt gegen Ende leicht an (siehe Abbildung A.3).

3. DenseASPP-Modell

Dice-Score:
Das DenseASPP-Modell zeigt eine stetige Zunahme des Trainings-Dice bis ca. 0,85. Der Validierungs-Dice stagniert bei etwa 0,80 (siehe Abbildung B.6). Die konstante Differenz zwischen den Kurven deutet auf moderates Overfitting hin.
Loss-Werte:
Der Trainings-Loss sinkt kontinuierlich, während der Validierungs-Loss ab Epoche 20 stagniert (siehe Abbildung A.6).

4. BiSeNetV2-Modell

Dice-Score:
Das BiSeNetV2-Modell zeigt die schwächste Generalisierungsleistung. Der Trainings-Dice erreicht ca. 0,8, während der Validierungswert deutlich darunter (0,6–0,7) liegt (siehe Abbildung B.5). Dies weist auf starkes Overfitting hin.
Loss-Werte:
Der Trainings-Loss fällt stetig, aber der Validierungs-Loss bleibt hoch und stagniert (siehe Abbildung A.5).

5. BASNet-Modell

Dice-Score:
Das BASNet-Modell erzielt sehr stabile und hohe Werte (Trainings-Dice ca. 0,85, Validierung ca. 0,80) (siehe Abbildung B.4). Die enge Übereinstimmung zwischen beiden Kurven spricht für eine gute Generalisierungsfähigkeit.

Loss-Werte:
Trainings- und Validierungs-Loss sinken beide schnell und konsistent. Die Validierungskurve verbleibt auf niedrigem Niveau (siehe Abbildung A.4).

4.3.4.1. Vergleich der Modelle

Die durchgeführten Experimente zeigten, dass das BASNet-Modell insgesamt die beste Performance bei der Segmentierung von Zähnen aufweist. Insbesondere erzielte BASNet die höchsten Werte beim Dice-Koeffizienten und beim F1-Score, was auf eine hohe Genauigkeit bei der Erfassung der anatomischen Grenzen hindeutet.

UNet und SegNet erzielten ebenfalls zufriedenstellende Ergebnisse und erwiesen sich als stabile und zuverlässige Modelle, wobei insbesondere UNet durch seine ausgewogene Leistung zwischen Precision und Recall überzeugte. SegNet zeichnete sich durch eine hohe Recall-Rate aus, was bedeutet, dass nur wenige relevante Pixel übersehen wurden, jedoch war die Präzision im Vergleich etwas geringer.

Die Modelle BiSeNet und DenseASPP erzielten hingegen etwas schwächere Ergebnisse. BiSeNet zeigte insbesondere bei der Dice- und Foreground-Accuracy Schwächen, was auf Probleme bei der genauen Abgrenzung der Zahnstrukturen hinweist. DenseASPP erreichte eine hohe Recall-Rate, litt jedoch unter einer vergleichsweise niedrigen Präzision, was zu einer insgesamt moderaten Leistung führte.

Das Mask R-CNN-Modell konnte aufgrund der in Abschnitt *„Ausführliche Analyse zum Scheitern des Mask R-CNN-Modells“* beschriebenen Gründe nicht erfolgreich evaluiert werden.

4.3.4.2. Analyse und Interpretation

Die deutlichen Leistungsunterschiede zwischen den Modellen könnten auf spezifische Merkmale der jeweiligen Modellarchitekturen zurückzuführen sein. Beispielsweise profitieren Architekturen wie BASNet von ihrer speziellen Struktur zur Grenzerkennung (*boundary-awareness*), die bei der Erkennung kleiner anatomischer Details wie Zähnen erhebliche Vorteile bietet. Die guten Ergebnisse von UNet und SegNet lassen sich durch ihre Encoder-Decoder-Strukturen erklären, die eine zuverlässige und detaillierte Rekonstruktion der Segmentierungsmaske ermöglichen.

Die vergleichsweise schwächere Leistung von BiSeNet könnte daran liegen, dass das Modell primär für Echtzeit-Anwendungen mit geringerer Auflösung optimiert wurde und somit Schwierigkeiten mit detaillierten anatomischen Strukturen aufweist. DenseASPP hingegen erzeugt zwar reichhaltige Kontextinformationen, könnte jedoch möglicherweise bei kleinen und komplexen Strukturen durch die hohe Anzahl von dichten Verbindungen Schwierigkeiten haben, klare Segmentierungsgrenzen zu bestimmen.

4.3.4.3. Limitationen und Verbesserungspotentiale

Die vorliegende Studie weist einige Limitationen auf, die bei der Interpretation der Ergebnisse berücksichtigt werden sollten:

- **Limitierte Datensatzgröße und -vielfalt**: Obwohl der verwendete Datensatz repräsentativ ist, könnte eine größere Datenmenge und eine noch breitere Variation der Daten die Robustheit und Generalisierungsfähigkeit der Modelle weiter erhöhen.
- **Technische Schwierigkeiten mit Mask R-CNN**: Das Scheitern des Mask R-CNN-Modells schränkt die Vollständigkeit der Ergebnisse etwas ein. Zukünftige Studien könnten dieses Modell durch erweiterte Anpassungen und umfangreiche Hyperparameteroptimierung erneut untersuchen.
- **Optimierungspotential bei der Vorverarbeitung**: Die Ergebnisse könnten möglicherweise durch weitere Optimierungen der Datenvorverarbeitung verbessert werden, wie beispielsweise verbesserte Normalisierungsmethoden oder erweiterte Augmentationstechniken.

4.4. Fazit und zukünftige Arbeiten

In dieser Arbeit wurden verschiedene Deep-Learning-Modelle zur automatisierten Segmentierung dentaler Strukturen aus tomographischen Bilddaten implementiert, trainiert und systematisch evaluiert. Hierfür wurde zunächst ein umfangreicher Datensatz genutzt, der aufgrund von technischen Herausforderungen von ursprünglich geplanten 3D- auf 2D-Slice-Daten reduziert wurde. Anschließend erfolgte eine detaillierte Bewertung unterschiedlicher Modellarchitekturen (UNet, SegNet, BASNet, BiSeNet, DenseASPP und Mask R-CNN) hinsichtlich ihrer Segmentierungsleistung, wobei der Dice-Koeffizient, Precision, Recall und F1-Score als zentrale Evaluationsmetriken verwendet wurden.

Die Ergebnisse zeigten, dass BASNet insgesamt die besten Resultate hinsichtlich Segmentierungsgenauigkeit und Robustheit lieferte, dicht gefolgt von UNet und SegNet. Die Modelle BiSeNet und DenseASPP wiesen dagegen moderatere Ergebnisse auf, vor allem aufgrund von Schwächen bei feinen anatomischen Details. Das Mask R-CNN-Modell konnte trotz intensiver Untersuchung aufgrund technischer Schwierigkeiten und mangelnder Eignung für die verwendeten dentalen Datensätze nicht erfolgreich eingesetzt werden. Ebenso zeigte sich, dass eine initial angedachte Fine-Tuning-Strategie gegenüber einer klassischen vollständigen Trainingsmethode keinen Vorteil brachte, weshalb letztere bevorzugt wurde.

Für zukünftige Forschungsarbeiten bieten sich mehrere Verbesserungsmöglichkeiten an. Eine Erweiterung und Diversifikation des Datensatzes hinsichtlich anatomischer Variationen und unterschiedlicher Aufnahmebedingungen könnte wesentlich zur Erhöhung der Generalisierungsfähigkeit beitragen. Insbesondere sollten hierbei gezielt mehr Bilddaten des Unterkiefers aufgenommen werden, um auch für diese Region eine zuverlässige Segmentierung zu ermöglichen, da der aktuelle Datensatz nur eine begrenzte Anzahl an Unterkieferbildern enthält. Weiterhin bietet es sich an, die vorgestellte 2D-Segmentierungsmethode systematisch auf alle Slices des ursprünglichen 3D-Datensatzes anzuwenden, um daraus eine vollständige, dreidimensionale Segmentierung der dentalen Strukturen zu rekonstruieren.

Zusätzlich könnte eine umfassende und systematische Hyperparameteroptimierung, insbesondere bei komplexeren Modellen wie Mask R-CNN, entscheidend zur weiteren Steigerung der Modellperformance beitragen. Die Erforschung neuerer, fortschrittlicherer Architekturen, wie beispielsweise Transformer-basierte Netzwerke, sowie der Einsatz semi-überwachter oder schwach überwachter Lernmethoden, versprechen eine weitere Verbesserung sowohl der Leistung als auch der Skalierbarkeit der Modelle. Abschließend sollte zukünftig gezielt die Robustheit der entwickelten Segmentierungsmodelle gegenüber typischen klinischen Artefakten, variierenden Aufnahmebedingungen

und Qualitätsunterschieden untersucht werden, um die Praxistauglichkeit und klinische Anwendbarkeit der Methoden nachhaltig zu erhöhen.

Insgesamt bildet diese Arbeit somit eine solide Grundlage für weiterführende Forschungsprojekte in der automatisierten Zahnsegmentierung und weist klare Perspektiven für zukünftige methodische und praktische Verbesserungen auf.

A. Anhang: Lernkurven

A.0.0.1. UNet

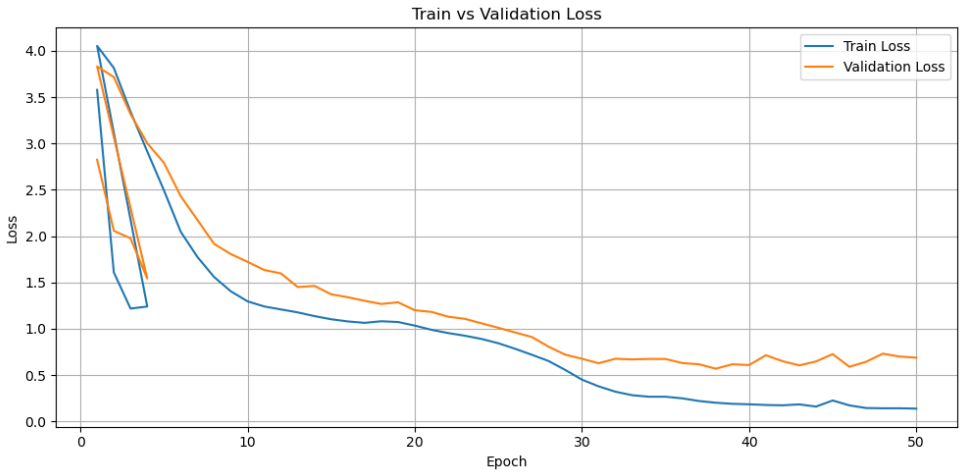


Abbildung A.1.: Lernkurve des Unet-Modell



Abbildung A.2.: Lernkurve des UNet-Modells mit Fine-Tunning

A.0.0.2. SegNet



Abbildung A.3.: Lernkurve des SegNet-Modells

A.0.0.3. BASNet

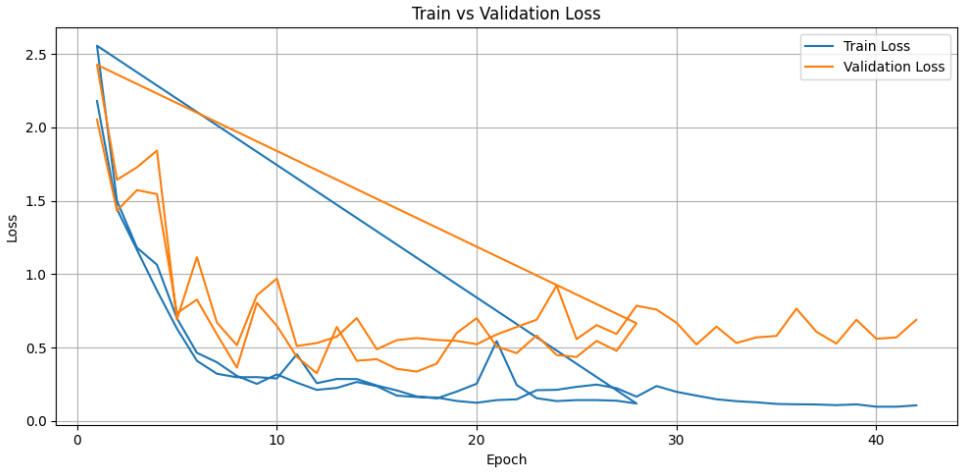


Abbildung A.4.: Lernkurve des BASNet-Modells

A.0.0.4. BiSeNetV2

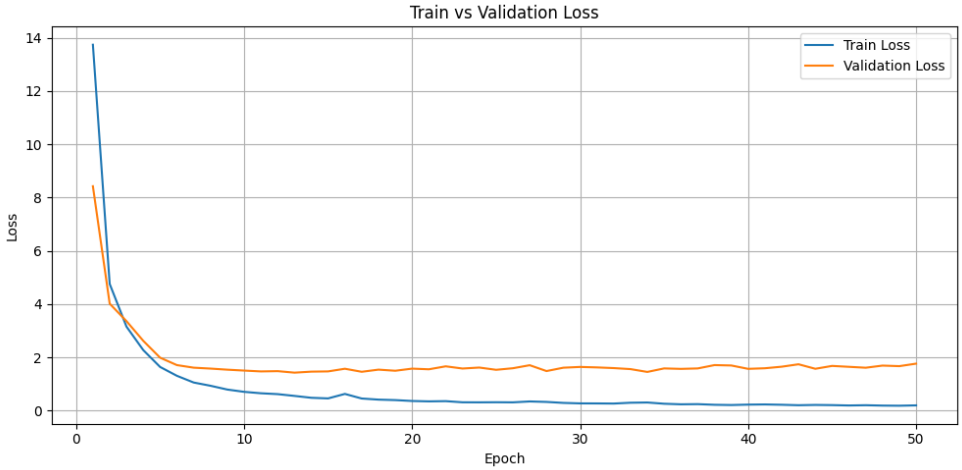


Abbildung A.5.: Lernkurve des BiSeNetV2-Modells

A.0.0.5. DenseASPP



Abbildung A.6.: Lernkurve des DenseASPP-Modells

B. Anhang: Dice Similarity Coefficient (DSC)

B.0.0.1. UNet

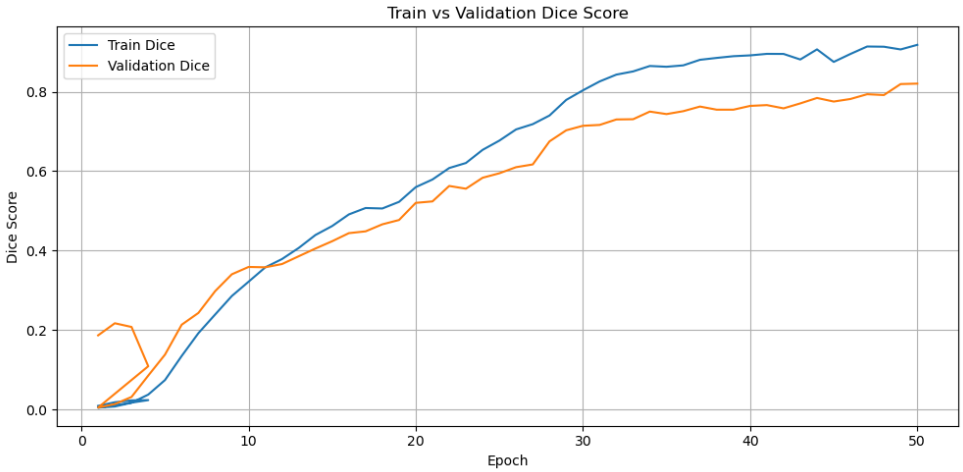


Abbildung B.1.: Dice Similarity Coefficient (DSC) des Unet-Modell

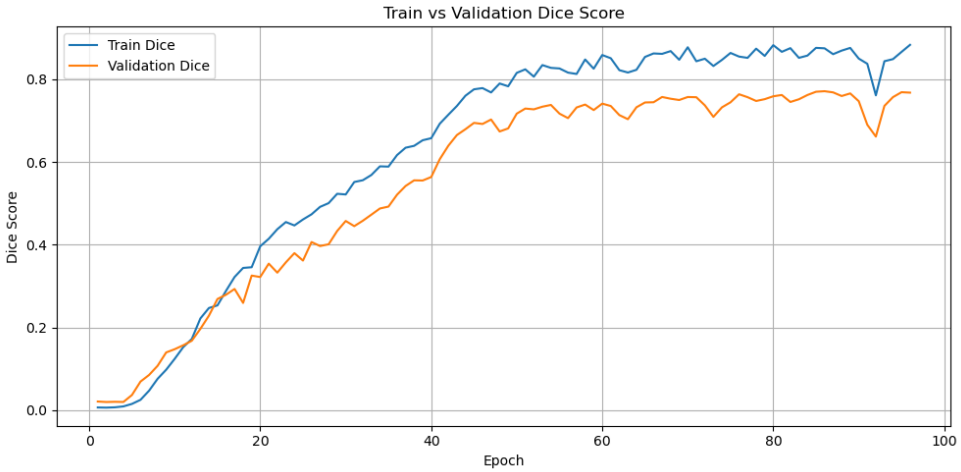


Abbildung B.2.: Dice Similarity Coefficient (DSC) des UNet-Modells mit Fine-Tunning

B.0.0.2. SegNet

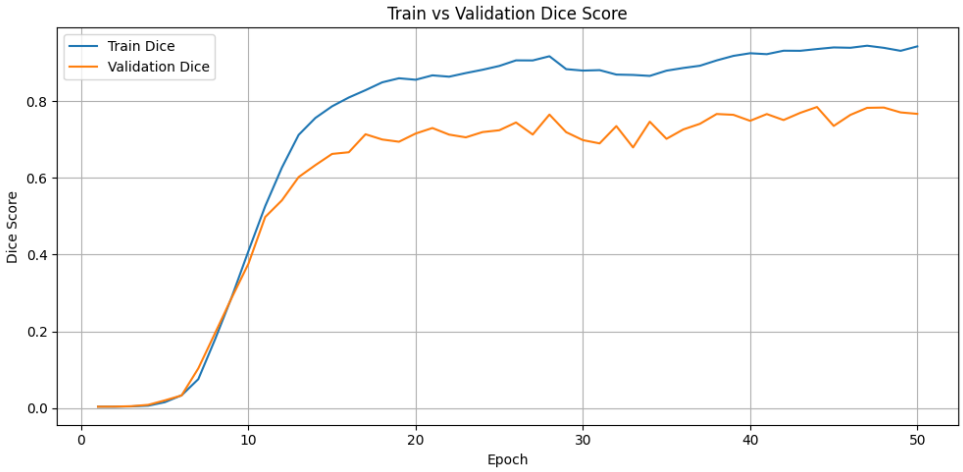


Abbildung B.3.: Dice Similarity Coefficient (DSC) des SegNet-Modells

B.0.0.3. BASNet

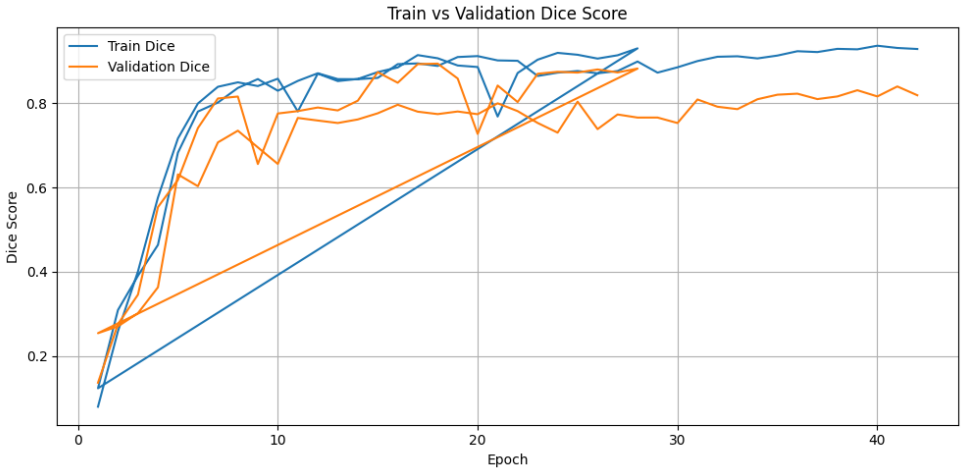


Abbildung B.4.: Dice Similarity Coefficient (DSC) des BASNet-Modells

B.0.0.4. BiSeNetV2

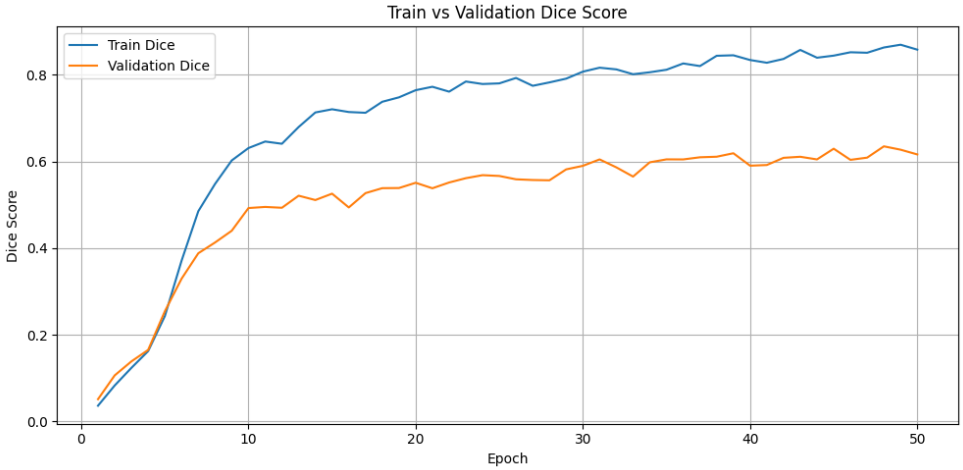


Abbildung B.5.: Dice Similarity Coefficient (DSC) des BiSeNetV2-Modells

B.0.0.5. DenseASPP

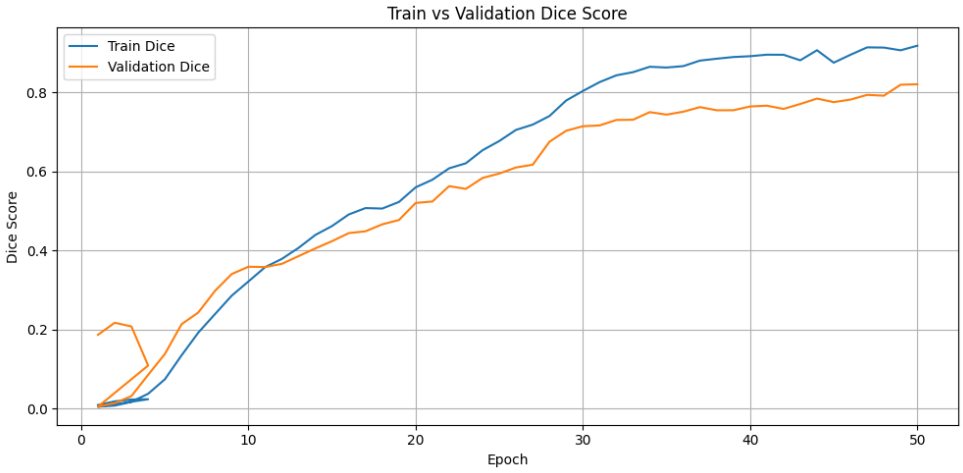


Abbildung B.6.: Dice Similarity Coefficient (DSC) des DenseASPP-Modells

Literaturverzeichnis

V. Badrinarayanan, A. Kendall, and R. Cipolla. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 39(12):2481–2495, 2017.

Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger. 3d u-net: learning dense volumetric segmentation from sparse annotation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2016: 19th International Conference, Athens, Greece, October 17-21, 2016, Proceedings, Part II 19*, pages 424–432. Springer, 2016.

A. Géron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow: Concepts, tools, and techniques to build intelligent systems.* Ö'Reilly Media, Inc.", 2022.

K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*, pages 2961–2969, 2017.

G. Jader, J. Fontineli, M. Ruiz, K. Abdalla, M. Pithon, and L. Oliveira. Deep instance segmentation of teeth in panoramic x-ray images. In *2018 31st SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI)*, pages 400–407. IEEE, 2018.

J.-H. Lee, D.-H. Kim, S.-N. Jeong, and S.-H. Choi. Detection and diagnosis of dental caries using a deep learning-based convolutional neural network algorithm. *Journal of dentistry*, 77:106–111, 2018.

L. Lumetti, V. Pipoli, F. Bolelli, E. Ficarra, and C. Grana. Enhancing Patch-Based Learning for the Segmentation of the Mandibular Canal. *IEEE Access*, pages 1–12, 2024. ISSN 2169-3536. doi: https://doi.org/10.1109/ACCESS.2024.3408629.

S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz, and D. Terzopoulos. Image segmentation using deep learning: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 44(7):3523–3542, 2021.

N. R. Pal and S. K. Pal. A review on image segmentation techniques. *Pattern recognition*, 26(9):1277–1294, 1993.

D. L. Pham, C. Xu, and J. L. Prince. Current methods in medical image segmentation. *Annual review of biomedical engineering*, 2(1):315–337, 2000.

B. Planche and E. Andres. *Hands-On Computer Vision with TensorFlow 2: Leverage deep learning to create powerful image processing apps with TensorFlow 2.0 and Keras*. Packt Publishing Ltd, 2019.

X. Qin, D.-P. Fan, C. Huang, C. Diagne, Z. Zhang, A. C. Sant'Anna, A. Suarez, M. Jagersand, and L. Shao. Boundary-aware segmentation network for mobile and web applications. *arXiv preprint arXiv:2101.04704*, 2021.

O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*, pages 234–241. Springer, 2015.

G. Silva, L. Oliveira, and M. Pithon. Automatic segmenting teeth in x-ray images: Trends, a novel data set, benchmarking and future perspectives. *Expert Systems with Applications*, 107:15–31, 2018.

M. Yang, K. Yu, C. Zhang, Z. Li, and K. Yang. Denseaspp for semantic segmentation in street scenes. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3684–3692, 2018.

C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, and N. Sang. Bisenet v2: Bilateral network with guided aggregation for real-time semantic segmentation. *International journal of computer vision*, 129:3051–3068, 2021.

Y. Zhao, P. Li, C. Gao, Y. Liu, Q. Chen, F. Yang, and D. Meng. Tsasnet: Tooth segmentation on dental panoramic x-ray images by two-stage attention segmentation network. *Knowledge-Based Systems*, 206:106338, 2020.